

PROJECT REPORT

STROKE PREDICTION USING HEALTH INDICATORS

**Submitted by:
ANURAG NEPAL
SAN FRANCISCO,CALIFORNIA**

**Submitted for
MATH 448
INTRODUCTION TO STATISTICAL LEARNING AND DATA MINING
SPRING 2025**

1. Executive Summary

Strokes represent a critical global health crisis, ranking as the second leading cause of death worldwide and a primary contributor to long-term disability. The ability to accurately predict stroke likelihood at an early stage is paramount for enabling timely medical interventions, potentially mitigating severe outcomes and improving patient quality of life. This project aimed to address this challenge by developing and evaluating robust machine learning models capable of predicting stroke occurrence based on readily available patient health indicators and demographic data. The core objectives included identifying key predictive features associated with stroke risk and determining the most effective modeling strategies to handle common data complexities.

The analysis utilized the Stroke Prediction Dataset from Kaggle, encompassing 5,110 patient records and 12 features spanning demographics, medical history (such as hypertension, heart disease, average glucose level, BMI), and lifestyle factors (including marital status, work type, smoking status). A significant characteristic of the dataset was its pronounced class imbalance, with only 4.87% of records corresponding to stroke occurrences, alongside 201 missing BMI values. Data preprocessing involved careful imputation of missing BMI values using an age-group and gender-based averaging technique and removal of a single anomalous gender entry. Exploratory Data Analysis (EDA) through visualizations underscored the strong influence of factors like age, existing hypertension, heart disease, average glucose levels, and smoking history on stroke risk.

To build effective predictive models, the study confronted the class imbalance primarily using the Synthetic Minority Oversampling Technique (SMOTE) and addressed potential multicollinearity and dimensionality using Principal Component Analysis (PCA). A suite of eight classification algorithms: Logistic Regression (LR), Linear Discriminant Analysis (LDA), Quadratic Discriminant Analysis (QDA), K-Nearest Neighbors (KNN), Decision Trees (DT), Random Forest (RF), Bagging, and Boosting (GBM) were systematically evaluated. Each model was tested under four distinct experimental scenarios: a baseline (No SMOTE, No PCA), SMOTE only, PCA only, and a combination of SMOTE and PCA. Model performance was assessed using 5-fold cross-validation, focusing on Area Under the ROC Curve (AUC), Sensitivity, Accuracy, and Specificity.

The results convincingly demonstrated the critical importance of addressing class imbalance. Models trained on the original, imbalanced data typically exhibited high (but misleading) accuracy with near-zero sensitivity, rendering them ineffective for practical diagnosis. The application of SMOTE dramatically improved sensitivity across most models. The findings highlight that models like LDA and Logistic Regression, when appropriately augmented with techniques like SMOTE, can offer a strong balance of predictive accuracy and sensitivity crucial for identifying high-risk individuals.

2. Description of the Problem

Strokes represent a significant global health challenge, being the second leading cause of death worldwide, responsible for approximately 11% of total deaths and a primary contributor to long-term disability (WHO). The ability to predict stroke risk at an early stage is crucial for enabling timely medical interventions, potentially saving lives and improving patient outcomes. This project's primary goal is to develop a robust model capable of predicting the likelihood of a patient experiencing a stroke using readily available health indicators and demographic information. Key research questions addressed include identifying the most influential features associated with stroke risk and understanding if strokes disproportionately affect specific demographic groups. The insights derived aim to support preventive healthcare strategies and enhance our understanding of stroke pathogenesis through data-driven analysis.

3. Dataset Description, Cleaning and Preparation

3.1 Dataset Description

The analysis is based on the [Stroke Prediction Dataset](#) from Kaggle which contains health records for 5,110 patients. It includes the following features:

- Demographics
 - id: unique identifier
 - gender: “Male”, “Female” or “Other”
 - age: age of the patient
- Medical History
 - hypertension: 0 if the patient doesn't have hypertension, 1 if the patient has hypertension
 - heart_disease: 0 if the patient doesn't have any heart diseases, 1 if the patient has a heart disease
 - avg_glucose_level: average glucose level in blood
 - bmi: body mass index
 - smoking_status: “formerly smoked”, “never smoked”, “smokes” or “Unknown”
- Lifestyle/Work
 - ever_married: “No” or “Yes”
 - work_type: “children”, “Govt_job”, “Never_worked”, “Private” or “Self-employed”
 - Residence_type: “Rural” or “Urban”
- Target Variable
 - stroke: 1 if the patient had a stroke or 0 if not

An initial exploration revealed a significant class imbalance: approximately 95.13% of patients did not have a stroke, while only 4.87% did. This imbalance is a critical consideration for model development.

stroke <fctr>	count <int>	percentage <dbl>
0	4861	95.127202
1	249	4.872798

Then, from the initial summary of the dataset prior to cleaning and transformation, age ranges from about 0.08 to 82 years with median age being 45, avg_glucose_level ranges from about 55.12 to 271.74 with median glucose level in blood being 91.89. BMI was read as a character, which had 201 NA values, was converted to numeric ranging from 10.30 to 97.60 . Rest of the variables are categorical.

3.2 Data Cleaning

The BMI column was originally imported as a character variable. After converting it to numeric, 201 missing values were identified.

```
bmi
Length:5110
Class :character
Mode :character
```

```
bmi
Min. :10.30
1st Qu.:23.50
Median :28.10
Mean :28.89
3rd Qu.:33.10
Max. :97.60
NA's :201
```

Since the dataset is quite small, we cannot just drop the NA values which account for nearly 4% of the dataset. So, these missing values were imputed by calculating the average BMI for each subgroup defined by age group("Adult", "Children", "Infant", "Middle Aged", "Senior", "Teenager") and gender.

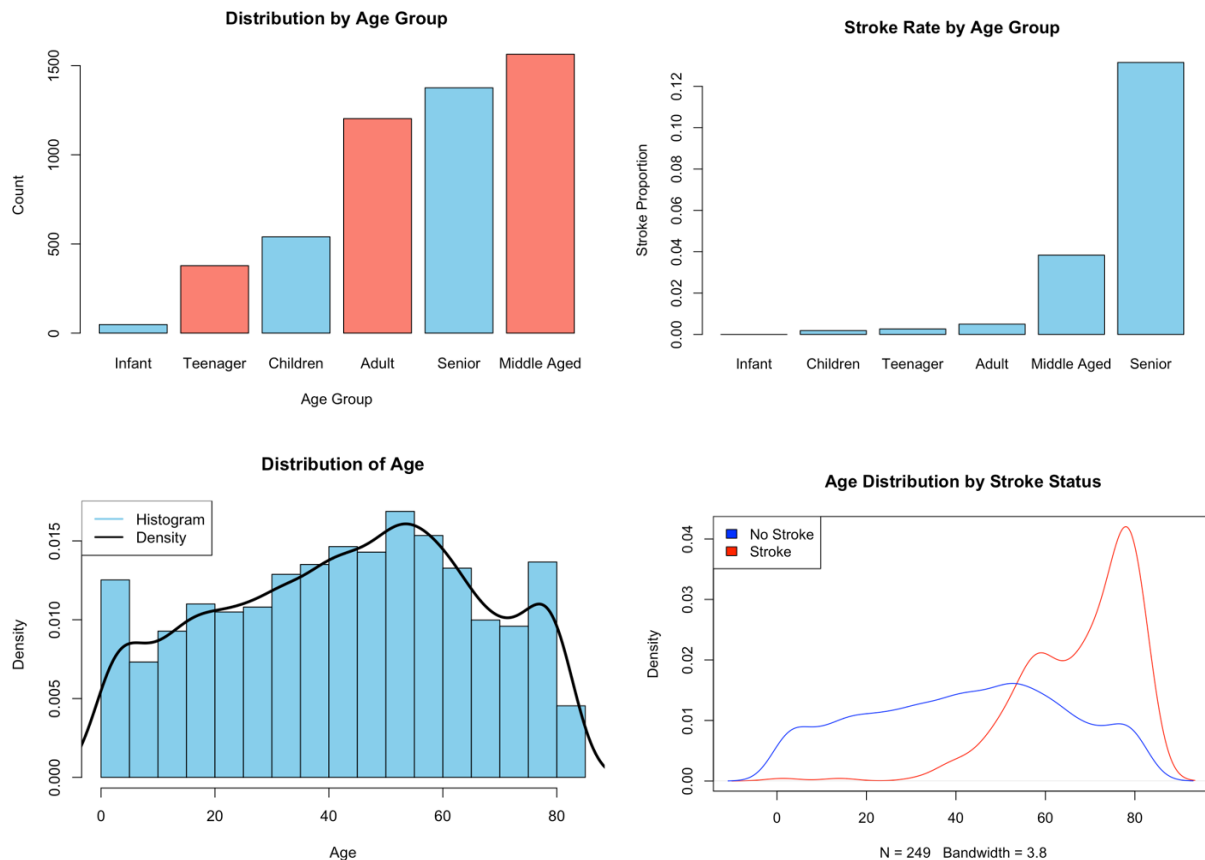
age_group	gender	avg_bmi
<chr>	<chr>	<dbl>
Adult	Female	29.68774
Children	Female	19.23281
Infant	Female	18.21250
Middle Aged	Female	31.40455
Senior	Female	29.90319
Teenager	Female	26.27750
Adult	Male	30.01599
Children	Male	19.67847
Infant	Male	18.43871
Middle Aged	Male	31.90985
Senior	Male	30.06266
Teenager	Male	25.67471

Also, since there was just one entry for "Other" gender, this row was removed from the dataset, to maintain consistency.

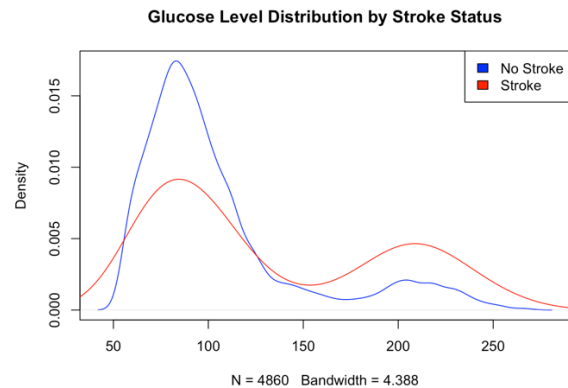
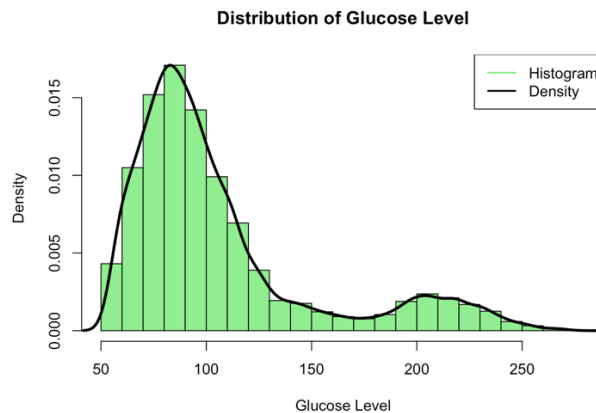
All relevant categorical variables (`ever_married`, `work_type`, `smoking_status`, `hypertension`, `heart_disease`) were converted to factor data types in R. The target variable `stroke` was also converted to a factor with levels "YES" and "NO".

4. Data Visualization and Findings

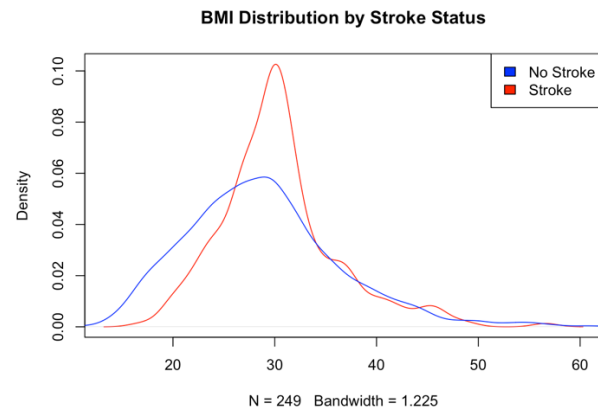
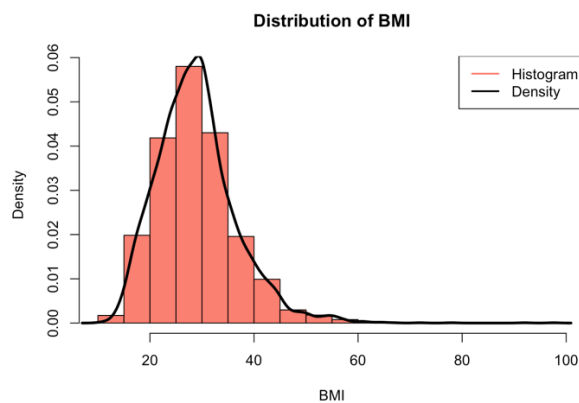
The histogram for the age groups reveals that the distribution of patients is concentrated in the 'Adult', 'Middle Aged', and 'Senior' categories. Importantly, density plots indicate that individuals experiencing a stroke tend to be older compared to the non-stroke group. So, Stroke rate increases dramatically with age, particularly in 'Middle Aged' and 'Senior' groups.



And the density plots confirm that the age distribution for stroke patients is heavily skewed towards older ages compared to non-stroke patients.



The glucose levels display a right-skewed distribution, with stroke patients showing a slightly higher average glucose level compared to non-stroke individuals. But there is not much of a difference between the distribution and the density plots.

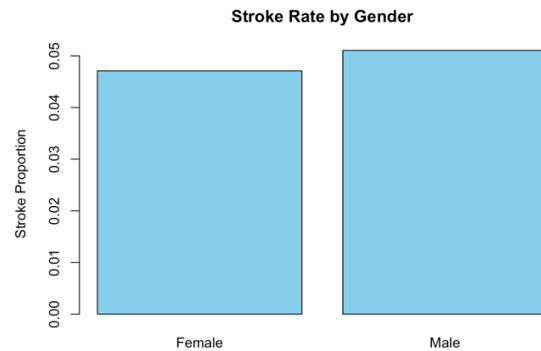
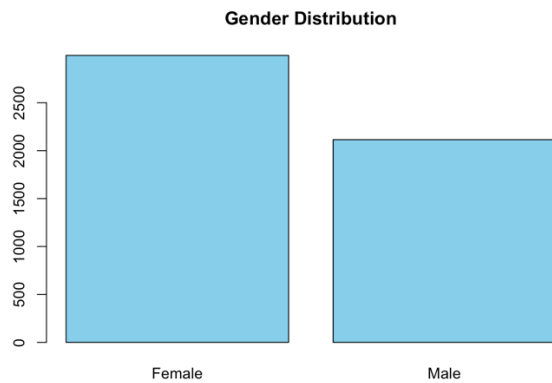


The BMI histogram also shows a relatively right-skewed spread across the dataset; the density plots show that BMI between 30 to 40 as a higher density for strokes but there is minimal difference between stroke and non-stroke groups which suggesting BMI alone is not a strong differentiating factor in this dataset.

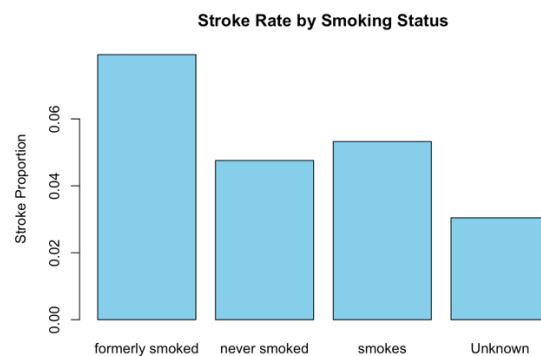
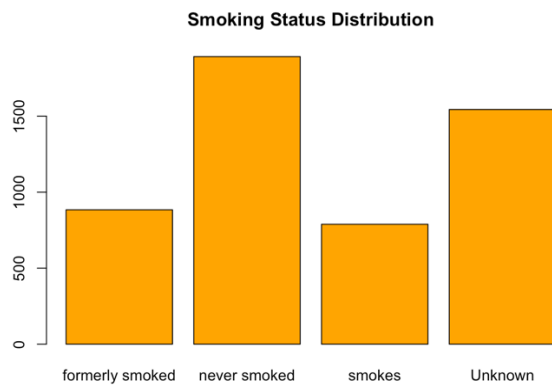
	age	avg_glucose_level	bmi	stroke
age	1.000000	0.2383228	0.33519923	0.24523876
avg_glucose_level	0.2383228	1.0000000	0.17267144	0.13199119
bmi	0.3351992	0.1726714	1.0000000	0.04332213
stroke	0.2452388	0.1319912	0.04332213	1.0000000

The correlation matrix among age, average glucose level, BMI, and stroke reveals, moderate positive correlation of age ($r \approx 0.245$) with stroke occurrence hinting that age is an indicator for stroke risk, weak positive correlation of average_glucose_level ($r \approx 0.132$) with stroke, and very low correlation of BMI ($r \approx 0.043$), suggesting it is not a strong indicator of stroke risk.

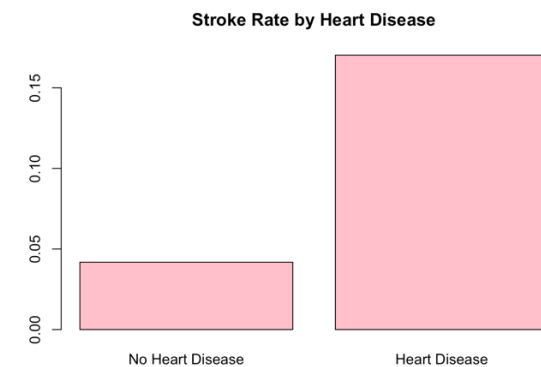
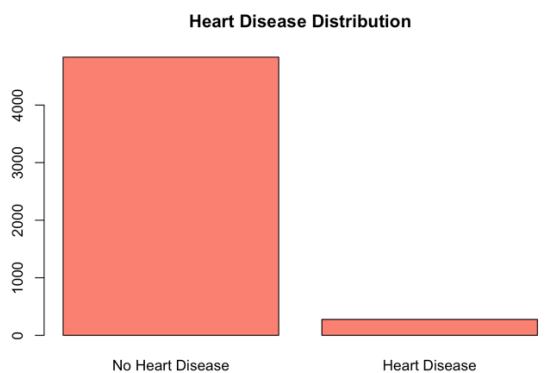
Now, for the categorical data, in terms of gender, the dataset comprised more females than males. However, the stroke rates between genders showed minimal differences, suggesting gender alone was not a major driver of stroke risk disparity in this sample.



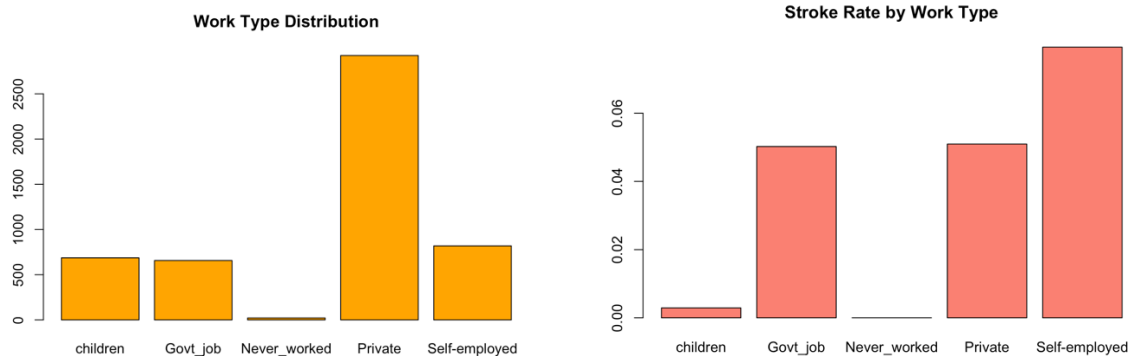
Former smokers exhibited the highest stroke rate suggesting that detrimental cardiovascular effects of smoking may persist even after people stop smoking and individuals who never smoked had a lower stroke rate but not enough difference from the people who actively smoke.



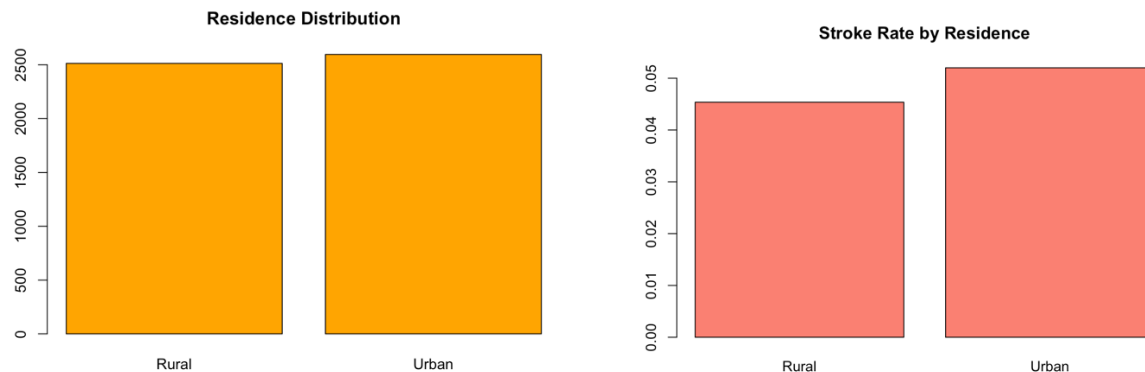
The presence of heart disease was strongly associated with an increased likelihood of stroke. Individuals with a history of heart disease had a threefold higher stroke rate compared to those without heart disease.



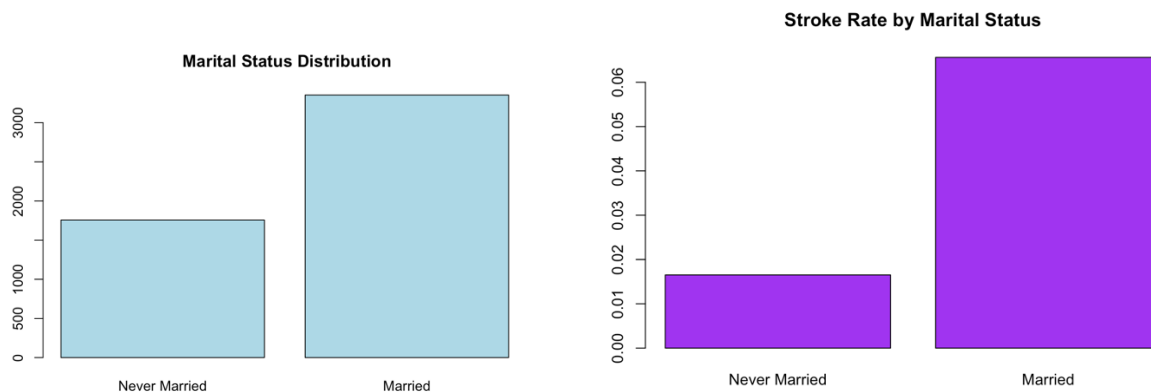
Individuals in Self-employed work categories exhibited higher chances of a stroke. This can be caused by stress in the job environments. Also, Private and Government jobs have a similar proportion to stroke risks.



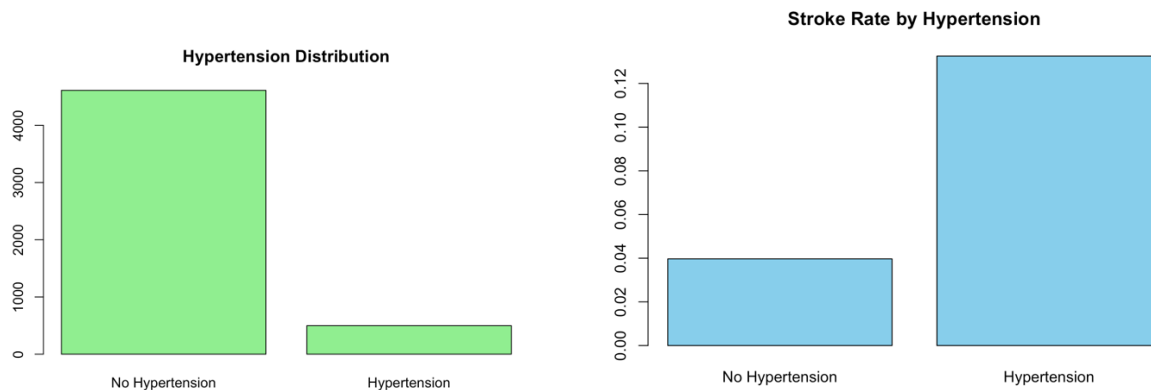
There is not enough discernable difference between the residential status, but Urban residents had a slightly higher stroke rate compared to Rural residents, possibly due to lifestyle differences such as diet, physical activity, or access to healthcare.



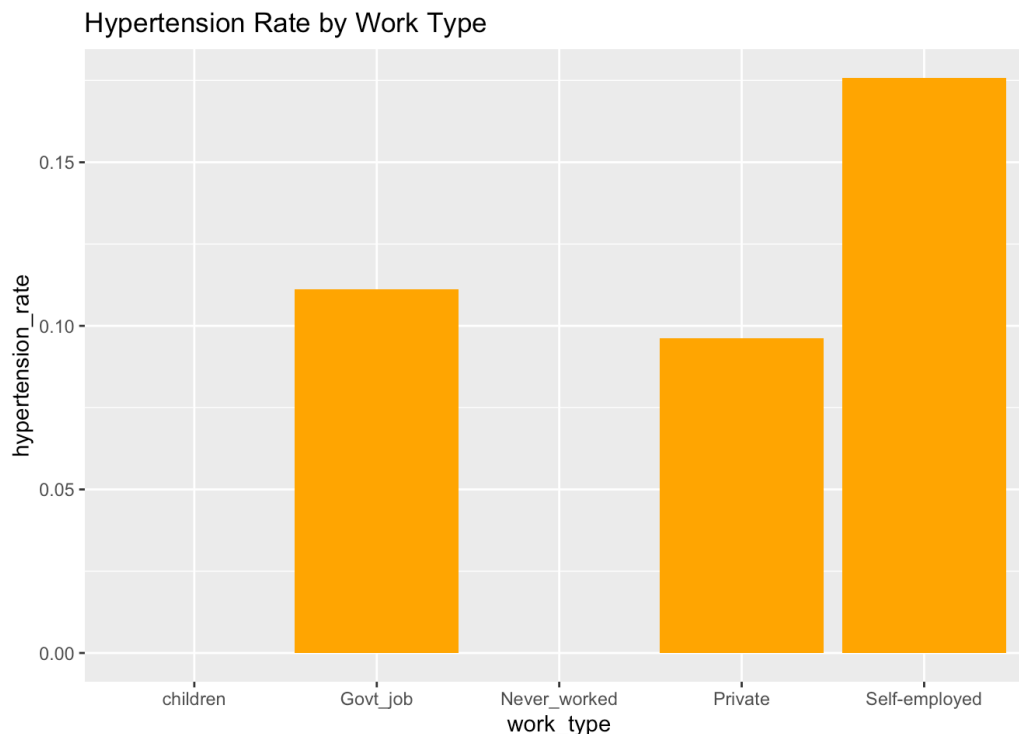
Married individuals also exhibited a higher stroke rate than those who were never married, which may reflect age-related factors or differences in stress levels.



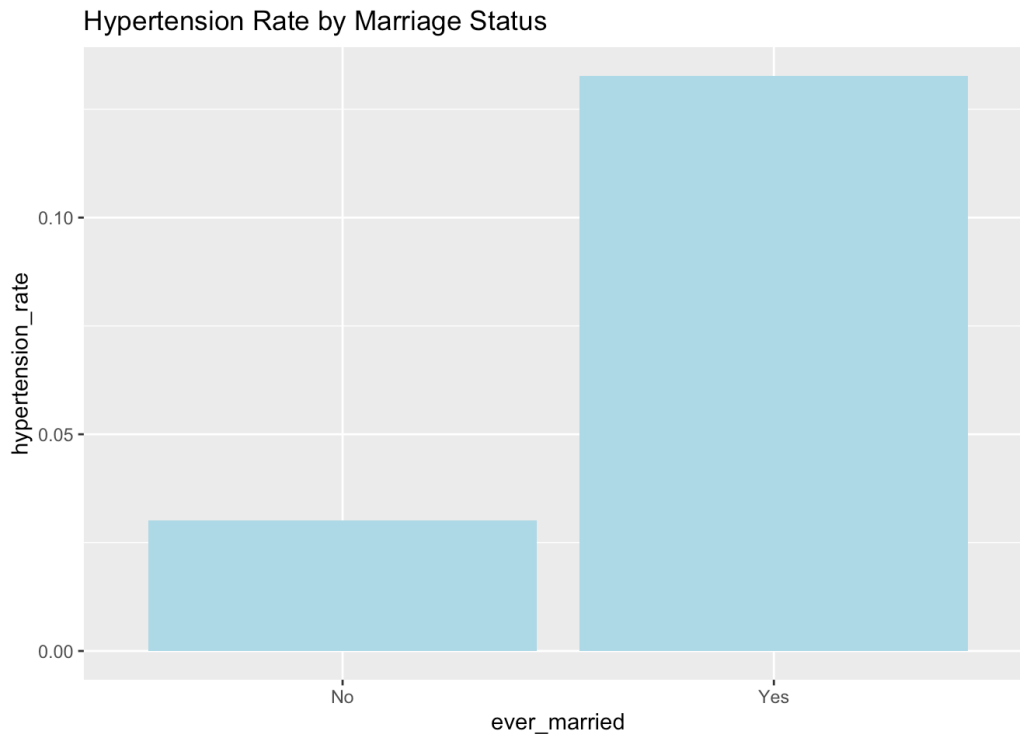
The presence of Hypertension was found to have a more than three times higher stroke rate compared to those without hypertension. Hypertension is caused by various factors which can be work, lifestyle choices, marriage status.



Individuals in Private and Self-employed work categories exhibited higher rates of hypertension, potentially due to occupational stress or irregular work schedules. This suggests that work-related factors may indirectly influence stroke risk through their impact on blood pressure and overall cardiovascular health.



Married Individuals exhibit a higher hypertension rate, which could be due to marriage related stress may indirectly influence stroke risk.



5. Methodology

5.1 Data Preprocessing for Modeling

After completing exploratory data analysis, the dataset is to be prepared for predictive modeling. The following steps are taken:

- **Variable Selection:** Non-predictive columns (id, age_group, Residence_type, gender) were removed. These variables were either identifiers("id") or show minimal correlation(age_group, Residence_type, gender) with stroke risk. Removing non-predictive variables reduces model complexity, mitigates overfitting, and improves interpretability. Categorical variables (ever_married, work_type, smoking_status, hypertension, heart_disease, stroke) were converted to factors.
- **Train-Test Split:** The dataset was split into 70% training and 30% testing sets using stratified sampling which enhances the generalizability of model evaluation by ensuring both sets reflect the population's class distribution. Due to the significant class imbalance in the 'stroke' variable (~5% strokes), simple random sampling could lead to training or test sets with very few or even no stroke cases, or proportions that don't reflect the overall data. Stratified sampling ensures that the proportion of stroke vs. non-stroke cases in both the training and testing sets mirrors the proportion in the original dataset.

5.2 Handling Class Imbalance and Dimensionality Reduction

As shown in the EDA there was class imbalance between the positive stroke and negative stroke cases as stroke is rare occurrence. Models trained on such data tend to become lazy. They can achieve high accuracy by simply predicting the majority class ("No Stroke") for all instances. We need models that are sensitive to the minority class i.e., good at correctly identifying the rare "Stroke" cases. This is often more important than overall accuracy in medical diagnostics. Two techniques are applied to address challenges in modeling:

- **SMOTE (Synthetic Minority Oversampling Technique):** SMOTE generates synthetic samples for the minority class by interpolating between existing minority samples and their nearest neighbors in feature space. For a stroke case with features (e.g., age=65, bmi=30, hypertension=1), SMOTE identifies its nearest neighbors, computes feature differences, and creates synthetic samples along the line segments connecting them. This increases the minority class size without duplicating existing data, reducing overfitting compared to simple oversampling. Also, SMOTE was applied to the training set only, preserving the test set's original distribution for unbiased evaluation.
- **Principal Component Analysis (PCA):** When predictor variables are highly correlated with each other. This can destabilize model coefficients and make interpretation difficult, particularly for models like QDA which rely on inverting covariance matrices. Applied to reduce dimensionality and mitigate multicollinearity among predictors specially during QDA. Multicollinearity occurs when predictors are correlated, causing instability in models like QDA, which assumes invertibility of the covariance matrix. Rank deficiency arises when the feature matrix is not full rank, preventing QDA from fitting properly.

5.3 Model Training and Scenarios

A diverse array of classification algorithms was selected to comprehensively evaluate predictive performance for stroke. Each model offers distinct advantages in handling data characteristics and underlying assumptions.

1. **Logistic Regression:** Logistic Regression is a parametric, linear classification model that predicts the probability of a binary outcome (e.g., stroke vs. no stroke) by modeling the log-odds as a linear combination of predictors. It is widely used in medical research due to its interpretability, computational efficiency, and robustness. LR assumes the log-odds of the outcome is linear in the predictors:

$$\log \left(\frac{P(\text{stroke} = 1 | X)}{P(\text{stroke} = 0 | X)} \right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p$$

where $X_1, X_2, \dots, X_p$ are predictors (e.g., age, hypertension), β_0 is the intercept, and $\beta_1, \dots, \beta_p$ are coefficients. The probability of stroke is computed using the logistic function:

$$P(\text{stroke} = 1 | X) = \frac{1}{\{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p)}\}}$$

Coefficients are estimated by maximizing the likelihood function.

2. **Linear Discriminant Analysis:** LDA is a parametric classification model that assumes classes follow multivariate normal distributions with equal covariance matrices. It seeks linear decision boundaries to separate stroke and non-stroke cases, making it effective for well-separated classes. LDA models the class-conditional densities of predictors (X) as multivariate normal, $P(X | \text{stroke} = k) \sim N(\mu_k, \Sigma)$. Where μ_k is the mean vector for class (k) (stroke or no stroke), and Σ is the shared covariance matrix. The decision boundary is derived by comparing the posterior probabilities $P(X | \text{stroke} = k)$, computed via Bayes' theorem. Linear boundaries are computationally efficient and interpretable. LDA performs well even with moderate deviations from normality, especially with large samples.

3. **Quadratic Discriminant Analysis:** QDA extends LDA by allowing class-specific covariance matrices, enabling quadratic decision boundaries. This flexibility suits complex datasets but increases sensitivity to multicollinearity and sample size. QDA assumes class-conditional densities as multivariate normal with distinct covariances, $P(X | \text{stroke} = k) \sim N(\mu_k, \Sigma_k)$. The quadratic terms arise from the inverse covariance matrices, allowing nonlinear boundaries. But QDA can fail due to rank deficiency, as the predictor matrix may not be full rank. PCA enables QDA to fit by providing uncorrelated PCs, though class imbalance persists.
4. **K-Nearest Neighbor:** KNN is a non-parametric, instance-based classifier that predicts an observation's class based on the majority class of its k-nearest neighbors in feature space. It is flexible but sensitive to dimensionality and scaling. KNN assigns a class by computing distances (e.g., Euclidean) between a test observation and all training observations, selecting the k-nearest neighbors, and taking a majority vote. The distance metric (e.g., Euclidean, Manhattan) and k determine the model's behavior.
5. **Decision Tree (DT):** A non-parametric model that creates a hierarchical, tree-like structure to make predictions. Internal nodes in the tree represent tests on specific features, branches represent the outcomes of these tests, and leaf nodes represent the final class labels (stroke or no stroke). The algorithm recursively splits the data based on features that best separate the classes, often using metrics like Gini impurity or information gain. Decision trees are highly interpretable and can handle both numerical and categorical data. However, they are prone to overfitting and can be unstable, meaning small changes in the data can lead to significantly different tree structures.
6. **Random Forest (RF):** An ensemble learning method that operates by constructing a multitude of decision trees at training time. For classification, the final prediction is determined by the majority vote of all individual trees. Each tree in the forest is trained on a bootstrap sample (random sample with replacement) of the training data. Furthermore, at each split in a tree, only a random subset of features is considered, which helps to decorrelate the trees and reduce variance. Random Forests are known for their high accuracy, robustness to overfitting, and ability to handle high-dimensional data and complex interactions. Their main drawback is reduced interpretability compared to a single decision tree.
7. **Bagging :** Bootstrap Aggregating is another ensemble technique that aims to reduce variance and prevent overfitting. Similar to Random Forest, it involves creating multiple bootstrap samples from the training data and training a base learner (typically a decision tree) on each sample. The predictions from these individual models are then aggregated, usually by majority voting for classification. Unlike Random Forest, standard bagging typically considers all features at each split when growing the trees.
8. **Boosting (Gradient Boosting Machine - GBM):** An ensemble technique that builds models sequentially, where each new model attempts to correct the errors made by the ensemble of previously trained models. GBM fits new models (typically decision trees) to the residuals (the differences between the actual values and the predictions) of the preceding models. This iterative process focuses on difficult-to-classify instances. Boosting methods, like GBM, are often capable of achieving very high accuracy and can model complex non-linear relationships. However, they can be more prone to overfitting

if not carefully tuned (e.g., learning rate, number of trees, tree depth) and can be computationally intensive.

To comprehensively evaluate the effectiveness of class imbalance handling and dimensionality reduction, four scenarios were tested across all the models.

1. **Baseline: No SMOTE, NO PCA:** The objective of this was to establish a performance benchmark using the raw, unmodified dataset. The model trains on the original imbalanced dataset (95% non-stroke, 5% stroke) and All features are retained without transformation.
2. **SMOTE Only: Class Balancing with SMOTE :** The objective of this was to mitigate class imbalance by synthetically generating stroke cases. SMOTE (Synthetic Minority Oversampling Technique) creates synthetic samples of the minority class (stroke) in the training set but the test set remains untouched to preserve real-world distribution.
3. **PCA Only: Dimensionality Reduction with PCA:** The objective of this was to simplify the feature space by retaining the most informative components. Principal Component Analysis (PCA) transforms original features into orthogonal components that explain maximum variance. The features are centered, scaled, and reduced to principal components (PCs). This is to make QDA feasible.
4. **SMOTE + PCA: Combined Class Balancing and Dimensionality Reduction:** The objective of this was to Leverage synergies between SMOTE and PCA for robust performance. SMOTE is applied first to balance classes. PCA then reduces dimensionality of the balanced dataset.

5.4 Cross-Validation and Evaluation Metrics

5-fold cross-validation was used (trainControl in caret) to provide a robust estimate of model performance. The training data is split into 5 equal "folds", and the model is trained on 4 folds and validated on the remaining 1-fold. This process is repeated 5 times, with each fold serving as the validation set once and then the performance metrics are averaged across the 5 folds. This provides a more robust estimate of model performance and helps prevent overfitting compared to a single train/validation split. For scenarios with SMOTE, sampling was applied within each cross-validation fold, i.e., only on the 4 training folds, not the validation fold for that iteration. This is crucial to avoid data leakage from the SMOTE process into the validation data.

The AUC-ROC measures a model's ability to distinguish between positive (stroke) and negative (no stroke) classes across all classification thresholds. It ranges from 0 to 1, with higher values indicating better performance:

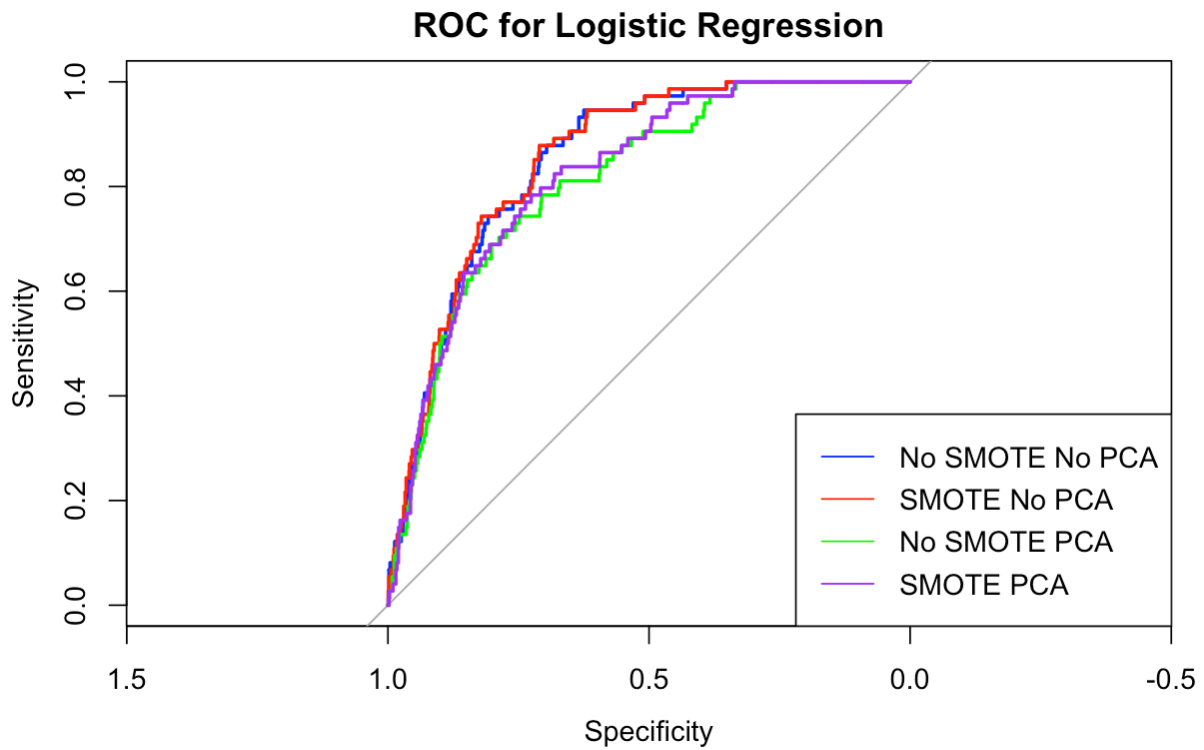
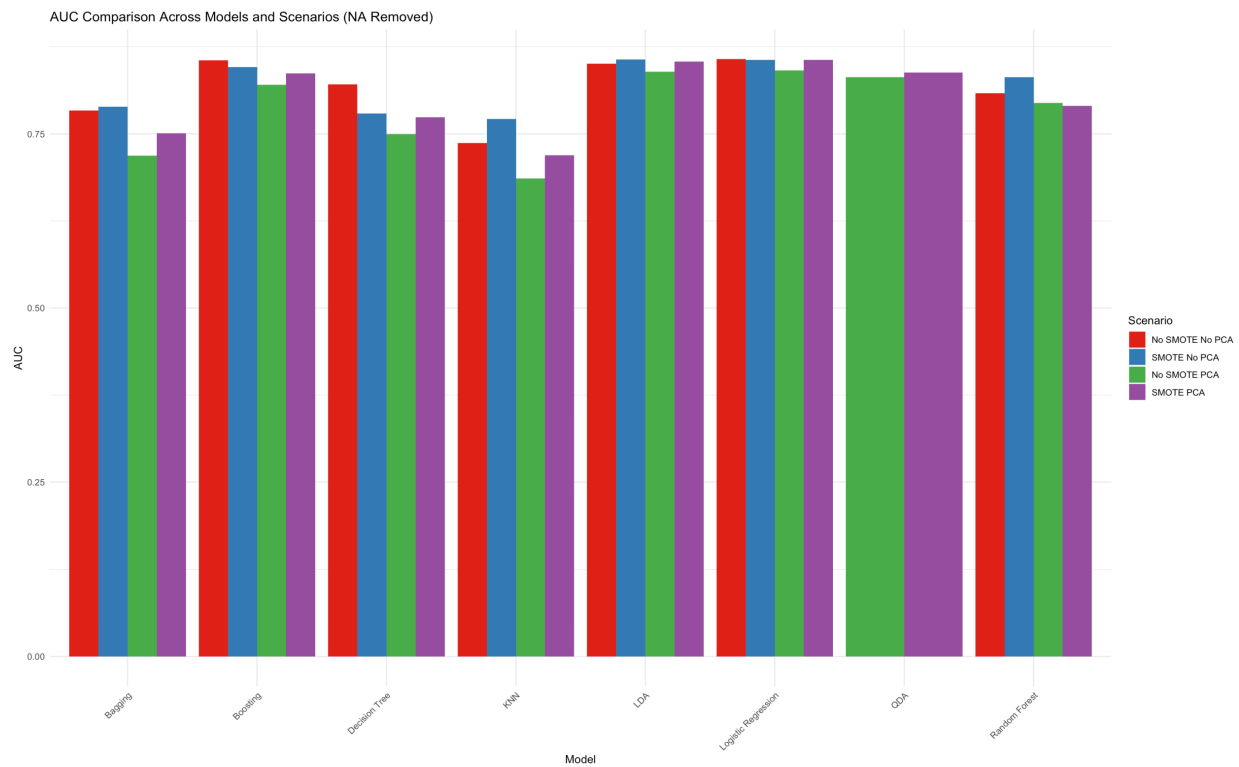
- 0.5: Random guessing (no discriminatory power).
- 0.7–0.8: Moderate discrimination.
- 0.8–0.9: Good discrimination.
- >0.9: Excellent discrimination.
- 0.0: Complete failure (e.g., model fails to fit).

For our case, AUC is particularly suitable due to class imbalance, as it evaluates performance across thresholds without relying on a fixed cutoff, unlike accuracy. Sensitivity (true positive rate) and specificity (true negative rate) are balanced in the ROC curve, making AUC a robust metric where detecting rare events is critical. In predicting a serious condition like stroke, Sensitivity

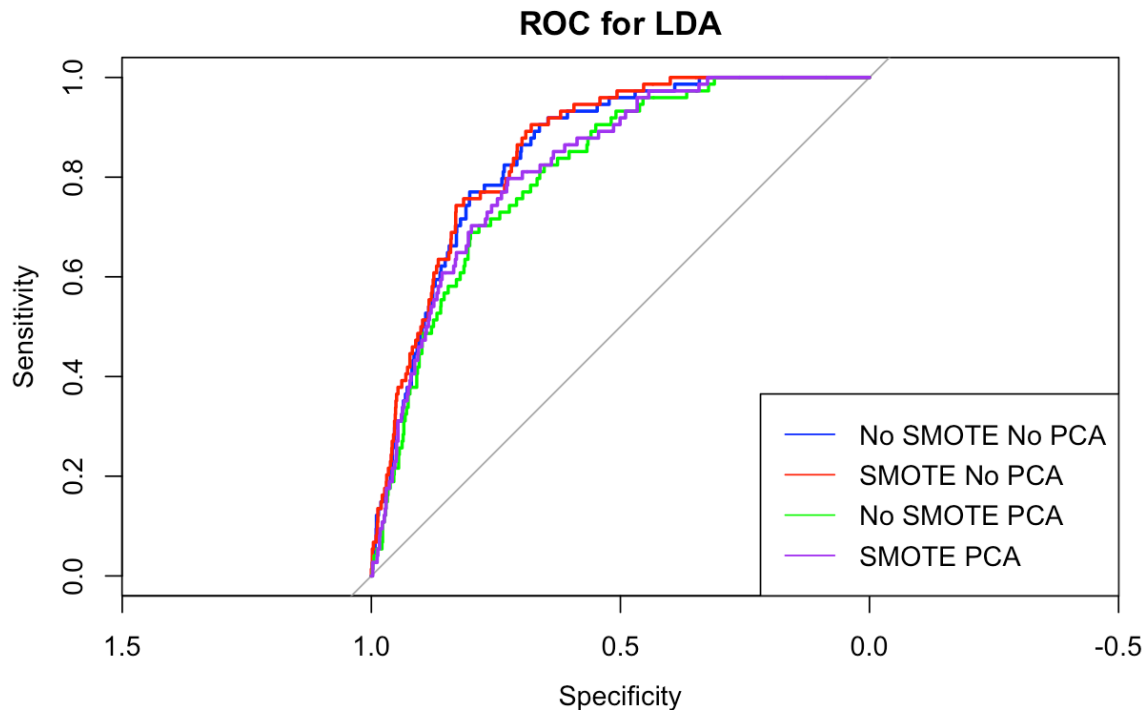
(correctly identifying those WITH stroke) is often prioritized, even if it means slightly lower Specificity (more false alarms among those WITHOUT stroke). Missing a stroke case (False Negative) has far more severe consequences than incorrectly flagging a healthy individual for further checks (False Positive).

6. Results

Model	Scenario	AUC	Accuracy	Sensitivity	Specificity
Logistic Regression	No SMOTE No PCA	0.857	0.9517	0	1
Logistic Regression	SMOTE No PCA	0.8558	0.7487	0.7973	0.7462
Logistic Regression	No SMOTE PCA	0.8411	0.9517	0	1
Logistic Regression	SMOTE PCA	0.8561	0.7448	0.8108	0.7414
LDA	No SMOTE No PCA	0.8505	0.9478	0.0405	0.9938
LDA	SMOTE No PCA	0.8566	0.7324	0.8108	0.7284
LDA	No SMOTE PCA	0.8389	0.9445	0.0405	0.9904
LDA	SMOTE PCA	0.8533	0.7265	0.8108	0.7222
QDA	No SMOTE PCA	0.8314	0.4563	0.9459	0.4314
QDA	SMOTE PCA	0.838	0.4458	0.973	0.4191
Decision Tree	No SMOTE No PCA	0.8211	0.9413	0.0405	0.987
Decision Tree	SMOTE No PCA	0.7791	0.9021	0.3378	0.9307
Decision Tree	No SMOTE PCA	0.7493	0.9419	0.0946	0.9849
Decision Tree	SMOTE PCA	0.7738	0.8003	0.5405	0.8134
Random Forest	No SMOTE No PCA	0.8084	0.9517	0	1
Random Forest	SMOTE No PCA	0.8313	0.9452	0.1757	0.9842
Random Forest	No SMOTE PCA	0.7943	0.9478	0.0405	0.9938
Random Forest	SMOTE PCA	0.7899	0.9178	0.2703	0.9506
Bagging	No SMOTE No PCA	0.7836	0.9452	0.0541	0.9904
Bagging	SMOTE No PCA	0.789	0.9328	0.1486	0.9726
Bagging	No SMOTE PCA	0.7189	0.9465	0.0541	0.9918
Bagging	SMOTE PCA	0.7506	0.9008	0.2568	0.9335
Boosting	No SMOTE No PCA	0.8557	0.951	0	0.9993
Boosting	SMOTE No PCA	0.8456	0.951	0	0.9993
Boosting	No SMOTE PCA	0.8204	0.9497	0.0135	0.9973
Boosting	SMOTE PCA	0.837	0.7409	0.8108	0.7373
KNN	No SMOTE No PCA	0.7369	0.9517	0.0135	0.9993
KNN	SMOTE No PCA	0.771	0.7911	0.6216	0.7997
KNN	No SMOTE PCA	0.686	0.9497	0	0.9979
KNN	SMOTE PCA	0.7194	0.8584	0.3649	0.8834

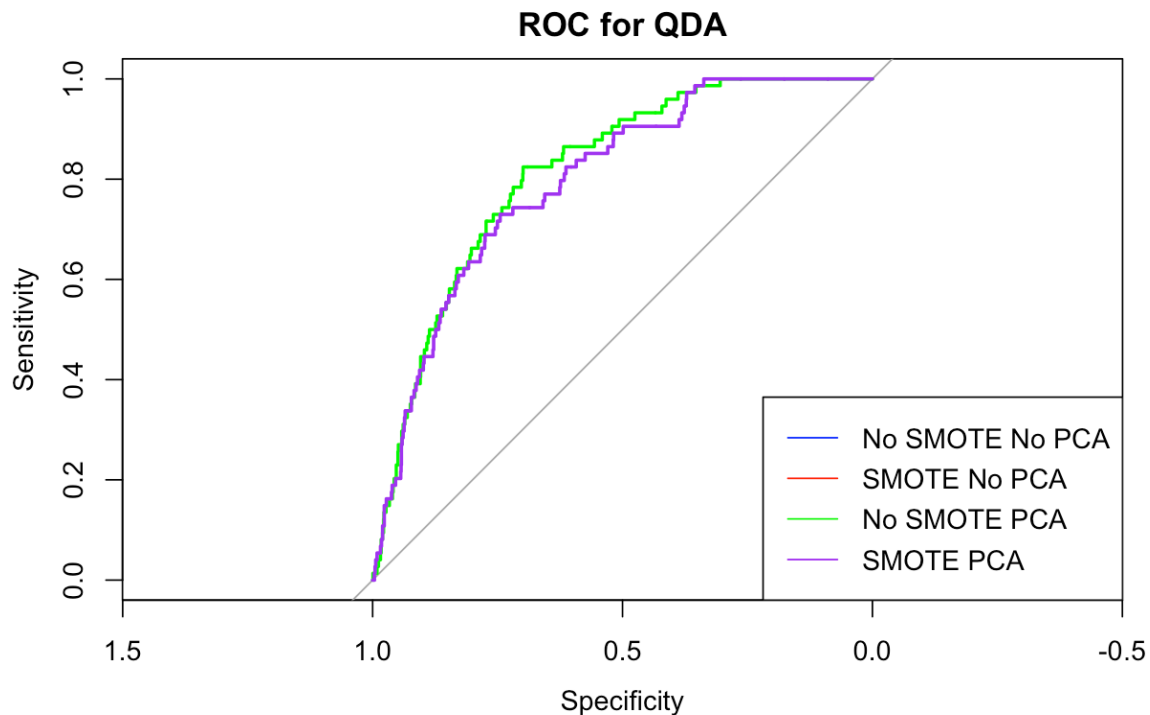


In the baseline scenario (No SMOTE, No PCA), Logistic Regression achieved a strong AUC of 0.8570, accompanied by a high accuracy of 0.9517. However, this high accuracy was misleading as the sensitivity was 0.0000, indicating the model failed to identify any true stroke cases, likely due to the class imbalance. When SMOTE was applied without PCA (SMOTE, No PCA), the AUC remained high at 0.8558. Critically, sensitivity surged to 0.7973, while accuracy adjusted to 0.7487. This demonstrates SMOTE's effectiveness in enabling the model to detect the minority (stroke) class, albeit with a trade-off in overall accuracy due to an increase in false positives. The application of PCA without SMOTE (No SMOTE, PCA) resulted in a slightly lower AUC of 0.8411, with accuracy remaining high at 0.9517 but sensitivity still at 0.0000. This suggests that PCA, while simplifying the feature space, might have discarded some predictive information or that the original features did not suffer from severe multicollinearity detrimental to LR. Finally, when both SMOTE and PCA were applied (SMOTE, PCA), the AUC was 0.8561, with sensitivity further improving slightly to 0.8108 and accuracy at 0.7448. Overall, for Logistic Regression, SMOTE was crucial for achieving meaningful sensitivity, and the model performed robustly in terms of AUC, particularly when class imbalance was addressed.



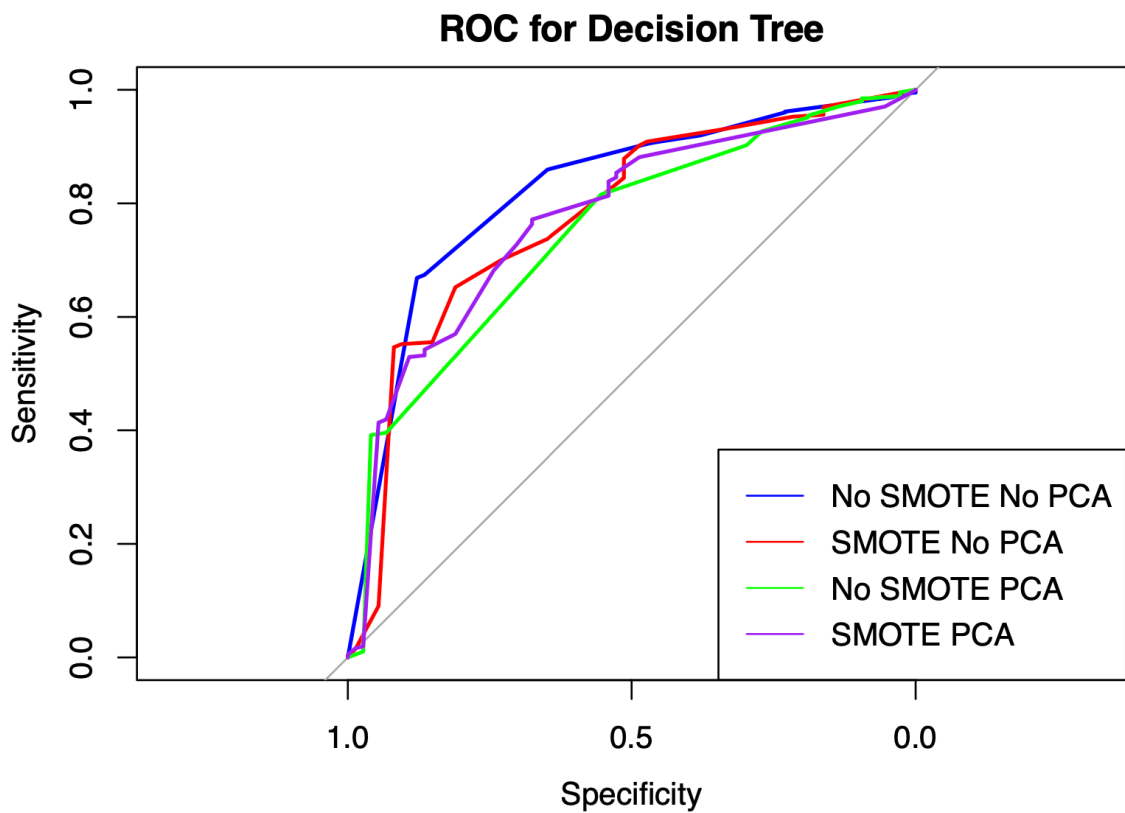
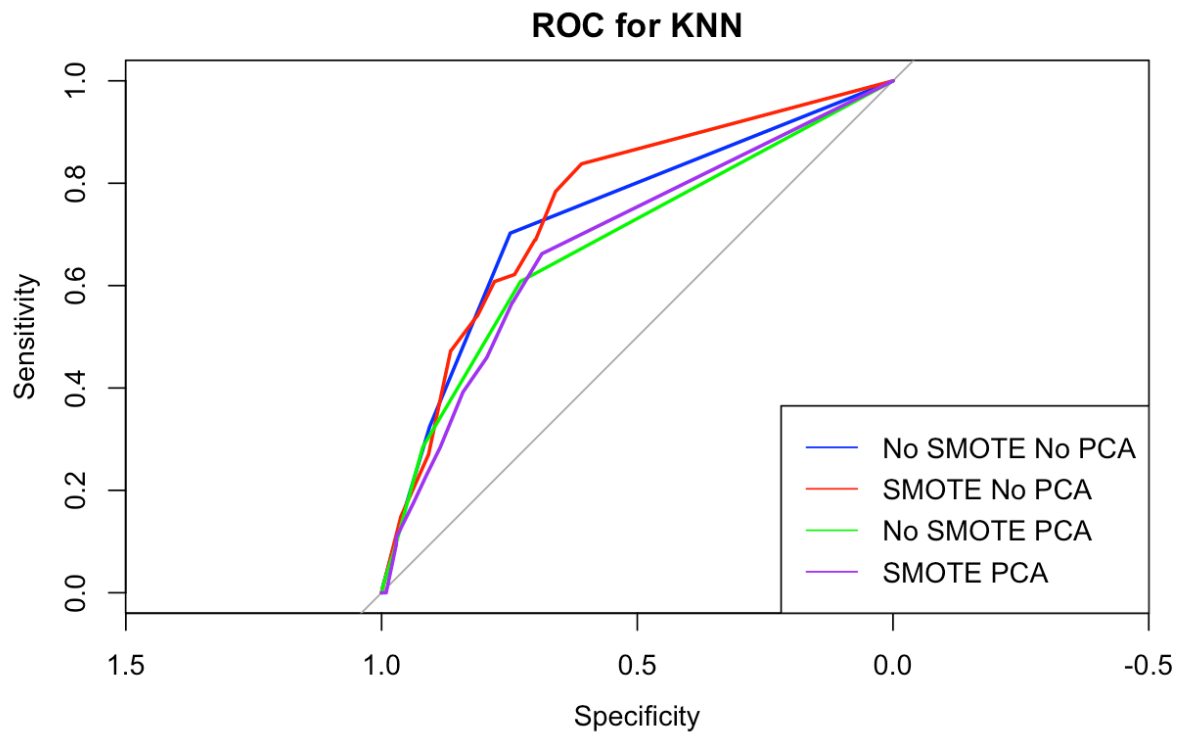
LDA, in its baseline configuration (No SMOTE, No PCA), yielded an AUC of 0.8505 and an accuracy of 0.9478. However, like LR, its sensitivity was very low at 0.0405, indicating poor performance in identifying actual stroke cases. The introduction of SMOTE (SMOTE, No PCA) led to the highest AUC observed across all models and scenarios, reaching 0.8566. This was coupled with a dramatic improvement in sensitivity to 0.8108, while accuracy became 0.7324. This highlights LDA's strong ability to leverage balanced class distributions for improved discrimination. When PCA was applied without SMOTE (No SMOTE, PCA), the AUC dropped to 0.8389, with accuracy at 0.9445 and sensitivity remaining low at 0.0405, suggesting that PCA might have removed variance important for LDA's linear separation. In the combined SMOTE and PCA scenario (SMOTE, PCA), the AUC was 0.8533, with sensitivity at 0.8108 and accuracy at 15

0.7265. For LDA, SMOTE was clearly beneficial, significantly boosting its ability to detect strokes while maintaining excellent discriminatory power.



QDA encountered issues in scenarios without PCA. As noted before, the model failed to train due to rank deficiency in the minority class when using the original feature set, even with SMOTE. Thus, results are only available for PCA-based scenarios. With PCA but without SMOTE (No SMOTE, PCA), QDA achieved an AUC of 0.8314. Interestingly, its sensitivity was very high at 0.9459, but this came at the cost of a very low accuracy of 0.4563, indicating a high number of false positives. When both SMOTE and PCA were applied (SMOTE, PCA), the AUC was 0.8380. Sensitivity further increased to an impressive 0.9730, the highest among all models. However, accuracy remained low at 0.4458. These results suggest that while PCA enables QDA to function, the model, especially with SMOTE, becomes highly sensitive to the minority class but struggles with overall correctness, likely due to the complexities of estimating separate covariance matrices with potentially noisy synthetic samples or a reduced feature space.

KNN, a non-parametric model, showed a baseline AUC of 0.7369 (No SMOTE, No PCA), accuracy of 0.9517, and very low sensitivity of 0.0135. The application of SMOTE (SMOTE, No PCA) improved the AUC to 0.7710, with accuracy becoming 0.7911 and a notable increase in sensitivity to 0.6216. This indicates that providing a more balanced neighborhood density helped KNN identify more stroke cases. When PCA was used without SMOTE (No SMOTE, PCA), performance degraded, with AUC dropping to 0.6860, accuracy at 0.9497, and sensitivity at 0.0000. This suggests that the reduced feature space from PCA might have been detrimental to KNN's distance-based calculations in the imbalanced setting. The combined SMOTE and PCA scenario (SMOTE, PCA) resulted in an AUC of 0.7194, accuracy of 0.8584, and sensitivity of 0.3649. For KNN, SMOTE provided a clear benefit to sensitivity, but its overall AUCs remained lower than the top linear models.

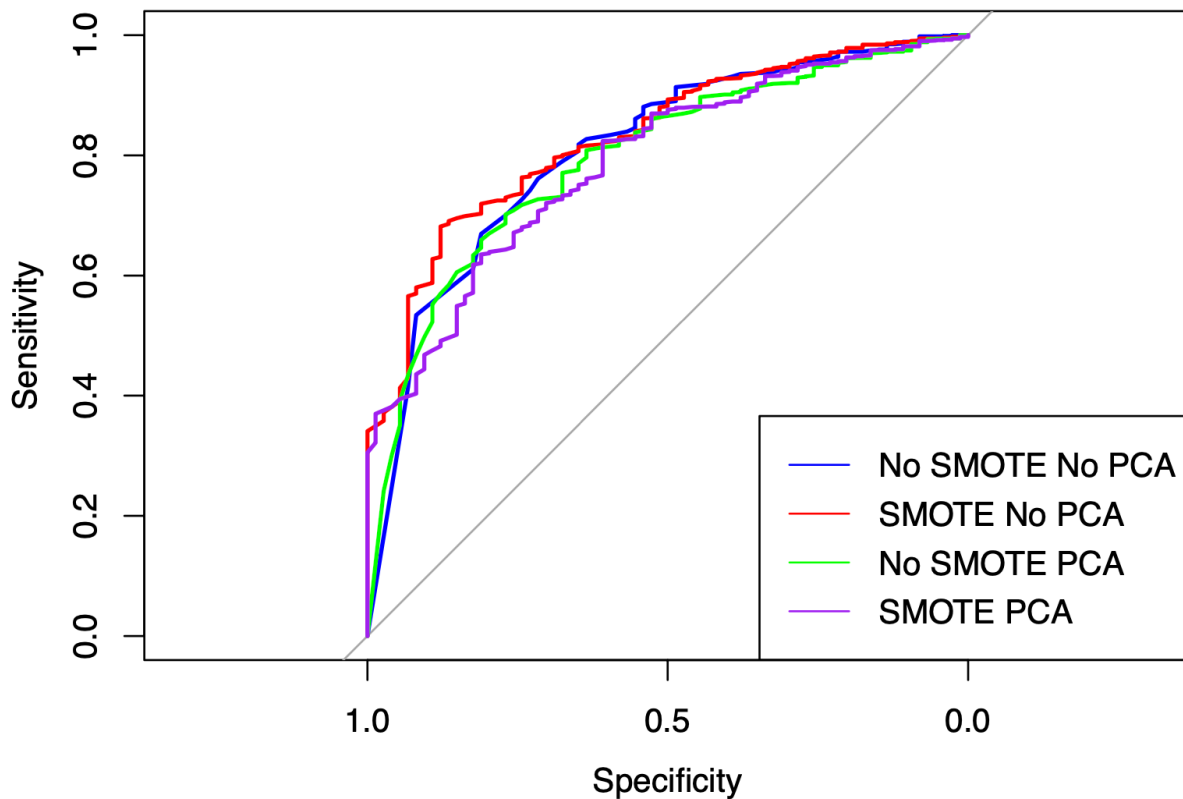


The Decision Tree model in its baseline state (No SMOTE, No PCA) produced an AUC of 0.8211, an accuracy of 0.9413, but a low sensitivity of 0.0405. Applying SMOTE without PCA (SMOTE,

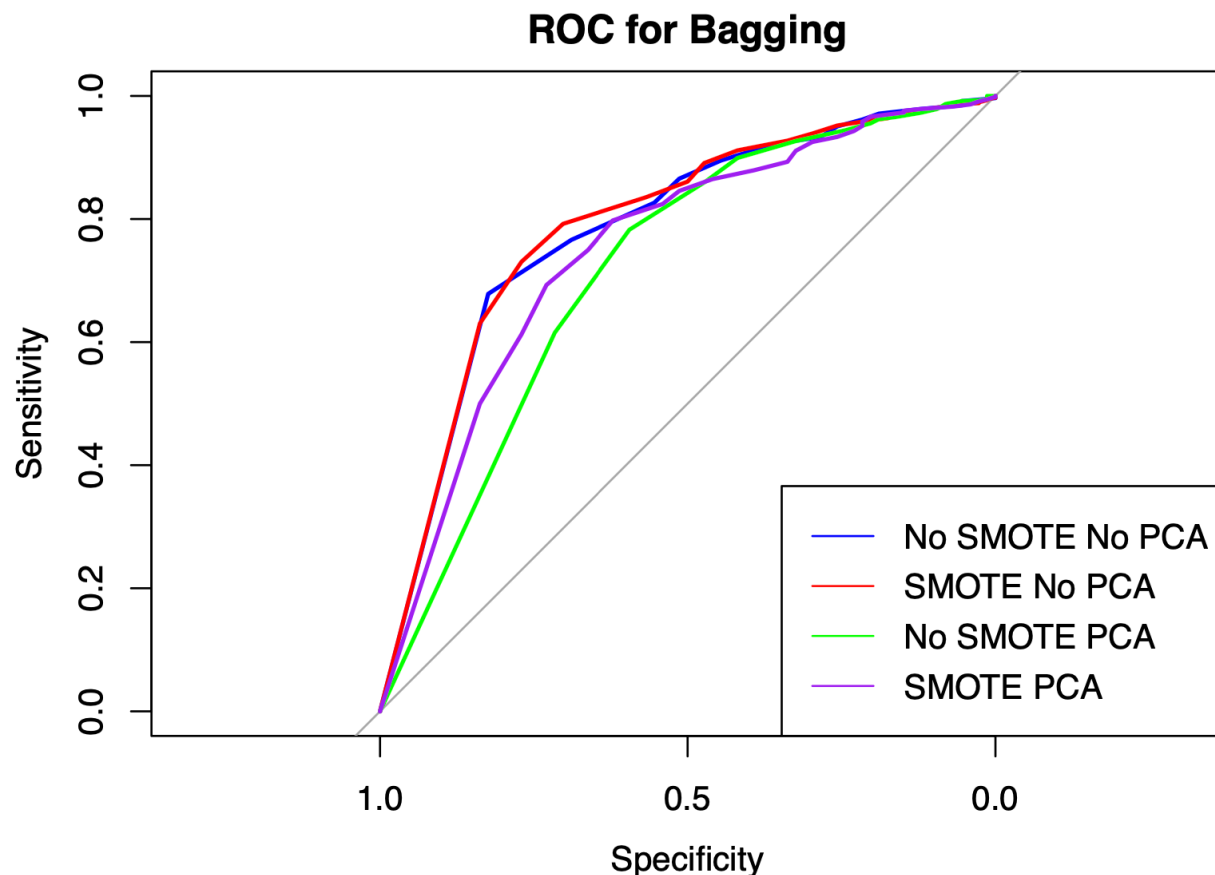
17

No PCA) led to a decrease in AUC to 0.7791, with accuracy at 0.9021 and an improved sensitivity of 0.3378. This suggests that while SMOTE helped in identifying more stroke cases, the standard decision tree might have struggled to optimally use the synthetic samples without overfitting or creating less generalizable rules. With PCA alone (No SMOTE, PCA), the AUC dropped further to 0.7493, accuracy was 0.9419, and sensitivity was 0.0946. The combination of SMOTE and PCA (SMOTE, PCA) resulted in an AUC of 0.7738, accuracy of 0.8003, and sensitivity of 0.5405. For Decision Trees, SMOTE did improve sensitivity, but the overall AUCs were generally lower than those of LR and LDA, indicating less robust discriminatory power in this application.

ROC for Random Forest

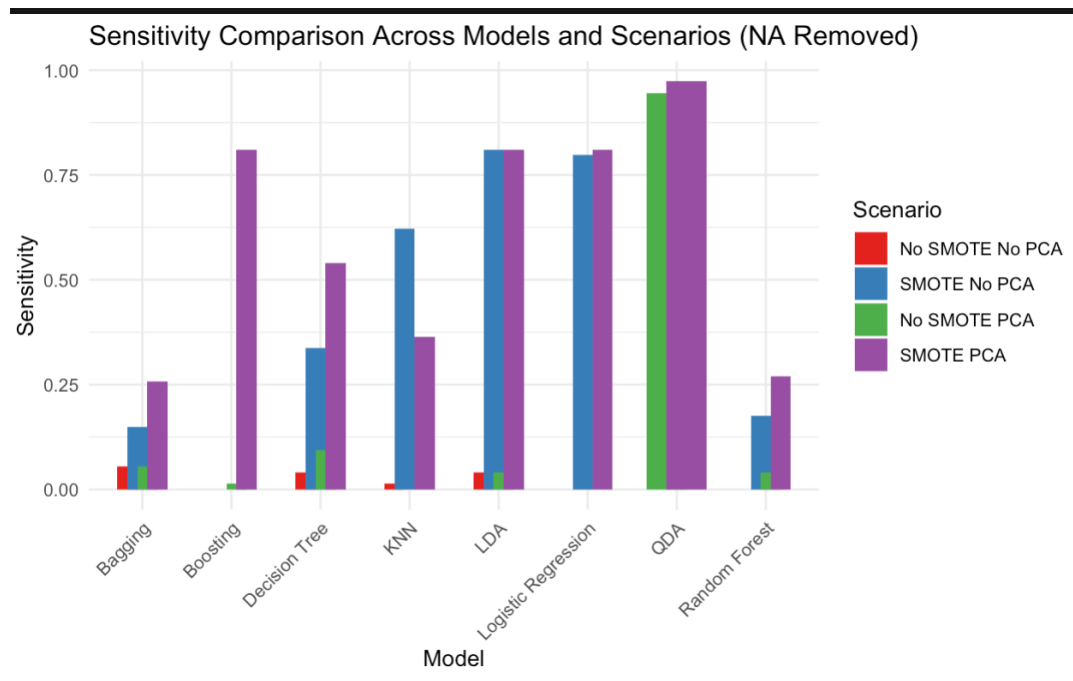
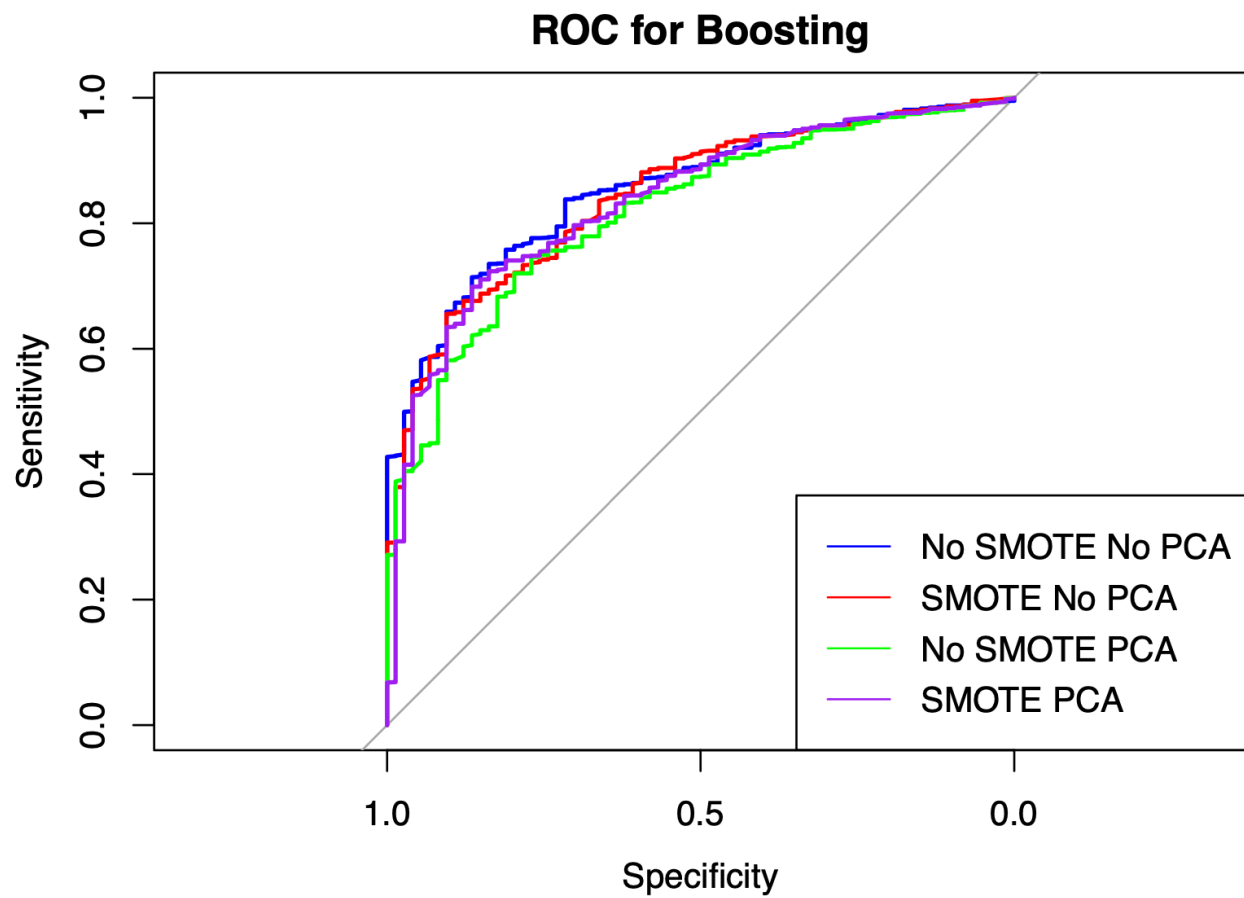


Random Forest, an ensemble of decision trees, started with an AUC of 0.8084 in the baseline scenario (No SMOTE, No PCA), an accuracy of 0.9517, but a sensitivity of 0.0000, failing to detect strokes. With SMOTE applied (SMOTE, No PCA), the AUC improved to 0.8313, accuracy was 0.9452, and sensitivity increased to 0.1757. While an improvement, this sensitivity was still relatively low compared to LR or LDA with SMOTE. The No SMOTE, PCA scenario yielded an AUC of 0.7943, accuracy of 0.9478, and sensitivity of 0.0405. When both SMOTE and PCA were used (SMOTE, PCA), the AUC was 0.7899, accuracy was 0.9178, and sensitivity reached 0.2703. Random Forest, despite its general robustness, did not achieve the high sensitivity levels seen with some other models in this specific imbalanced context, even with SMOTE, suggesting that further tuning or different feature representations might be needed.



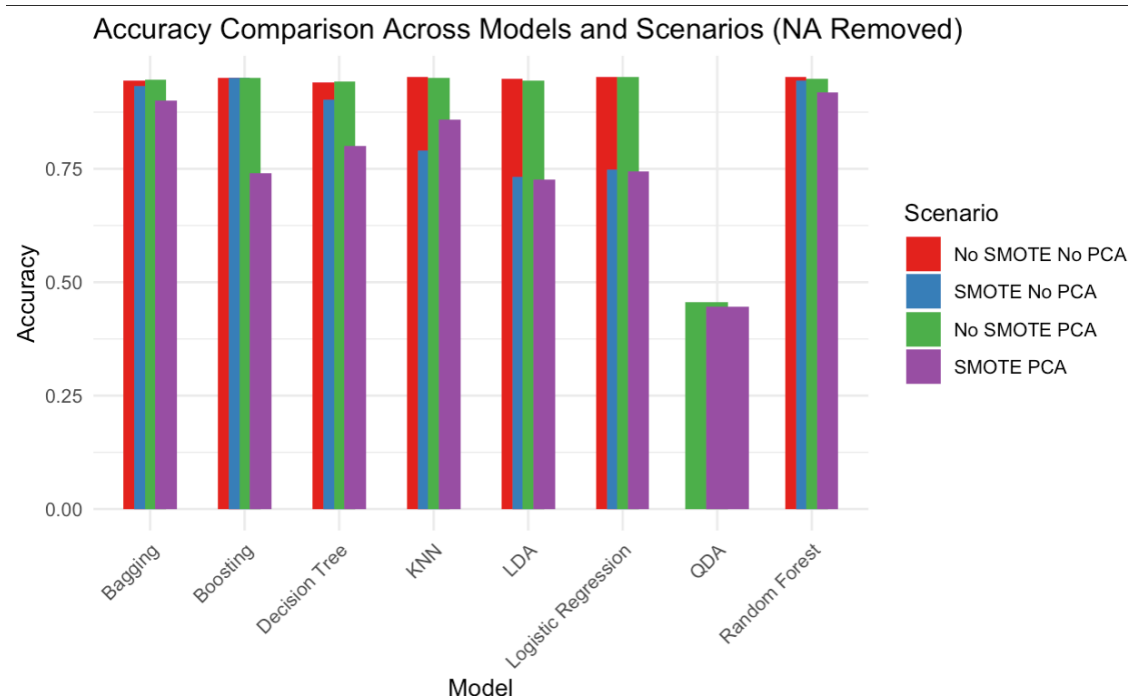
The Bagging model, similar in principle to Random Forest, showed an AUC of 0.7836, accuracy of 0.9452, and sensitivity of 0.0541 in the baseline (No SMOTE, No PCA) scenario. Applying SMOTE (SMOTE, No PCA) resulted in a slight AUC improvement to 0.7890, an accuracy of 0.9328, and an increased sensitivity of 0.1486. Using PCA without SMOTE (No SMOTE, PCA) led to a lower AUC of 0.7189, accuracy of 0.9465, and sensitivity of 0.0541. The combination of SMOTE and PCA (SMOTE, PCA) yielded an AUC of 0.7506, accuracy of 0.9008, and sensitivity of 0.2568. Like Random Forest, Bagging's sensitivity improvements with SMOTE were modest, and its overall AUCs were not among the top performers.

The Boosting model (GBM) demonstrated a high baseline AUC of 0.8557 (No SMOTE, No PCA) with an accuracy of 0.9510, but, critically, a sensitivity of 0.0000. This highlights a common pitfall with imbalanced data where high AUC can mask poor minority class detection. Surprisingly, applying SMOTE without PCA (SMOTE, No PCA) did not improve sensitivity for GBM in these results, which remained at 0.0000, with an AUC of 0.8456 and accuracy of 0.9510. This particular outcome for GBM with SMOTE alone is unusual and might point to issues with the default tuning parameters or interaction with the caret SMOTE implementation for this specific model. With PCA alone (No SMOTE, PCA), the AUC was 0.8204, accuracy 0.9497, and sensitivity was minimal at 0.0135. However, the combination of SMOTE and PCA (SMOTE, PCA) significantly changed GBM's behavior: the AUC was 0.8370, accuracy adjusted to 0.7409, and sensitivity markedly improved to 0.8108. This suggests that for GBM, the combination of class balancing and dimensionality reduction was necessary to achieve good sensitivity.



SMOTE dramatically increased Sensitivity for most models. For example, Logistic Regression's Sensitivity went from 0% to ~80-81%, and LDA's from ~4% to ~81% when SMOTE was applied. Accuracy often decreased with SMOTE, as models made more false positives to identify more true positives (strokes). This is an acceptable trade-off in medical diagnosis because without addressing class imbalance, these models are practically useless for the primary goal of identifying high-risk individuals for stroke. Boosting, for example, showed high AUC without SMOTE but had 0 sensitivity. Later, with SMOTE we tradeoff some accuracy for Sensitivity.

Reference			Reference		
Prediction	YES	NO	Prediction	YES	NO
YES	0	0	YES	59	370
NO	74	1458	NO	15	1088
Accuracy : 0.9517			Accuracy : 0.7487		



PCA is essential for QDA to resolve rank deficiency, enabling AUCs of 0.797–0.815. However, for models like LR and LDA, PCA often led to slightly lower AUCs compared to scenarios without PCA, suggesting some loss of information.

So, based on the analysis, several modeling strategies showed promise in predicting stroke by balancing overall discriminatory power (AUC) and the critical ability to correctly identify stroke cases (Sensitivity). Linear Discriminant Analysis (LDA) when combined with SMOTE (but without PCA) emerged as a particularly strong candidate, achieving a high AUC of approximately 0.857 and a robust sensitivity of around 0.8108, while also maintaining a reasonable specificity of about 0.7284. Logistic Regression, when used with SMOTE (irrespective of PCA application),

also delivered commendable results with a comparable AUC near 0.856 and sensitivity in the range of 0.80-0.81. Notably, Quadratic Discriminant Analysis (QDA), when used with both SMOTE and PCA, achieved the highest sensitivity at 0.9730; however, this came with a trade-off of a slightly lower AUC (0.8380) and reduced specificity (0.4191), suggesting that while it was excellent at catching nearly all stroke cases, it might also lead to more false alarms for healthy individuals. This makes the QDA configuration potentially valuable in clinical scenarios where the priority is to minimize missed diagnoses above all else.

7. Conclusion and Future Work

This project successfully demonstrated that machine learning models can predict stroke risk with reasonable accuracy when challenges like class imbalance are appropriately addressed. The analysis reaffirmed that age, hypertension, and heart_disease are dominant risk factors, with avg_glucose_level, smoking_status (especially "formerly smoked"), and work_type also contributing.

The highly skewed nature of the dataset (few stroke cases) poses a significant hurdle for standard classification algorithms. SMOTE proved to be an invaluable tool. It dramatically improved the models' ability to detect the minority class (stroke), as evidenced by the substantial gains in Sensitivity across multiple models. Without such balancing techniques, models are prone to high accuracy but are practically useless for identifying at-risk individuals.

PCA was indispensable for Quadratic Discriminant Analysis. QDA failed to run on the original feature set (even with SMOTE) due to rank deficiency. PCA, by creating uncorrelated principal components, resolved this issue and allowed QDA to be trained and evaluated. But, for other models like Logistic Regression and LDA, PCA did not consistently improve (and sometimes worsened) performance (AUC) compared to using the original features, especially when SMOTE was also applied. This suggests that the original features, despite potential correlations, held valuable information that was partially lost or obscured during the dimensionality reduction process for these specific models.

Future Work:

- **Advanced Model Exploration:** Dive deeper into models like XGBoost, LightGBM, or CatBoost, which are known for their high performance or explore SVMs with various kernels. Or even Neural Networks might give better results
- **Rigorous Hyperparameter Tuning:** The `tuneLength` parameter in `caret` provides basic tuning. Employ more exhaustive grid searches or random searches (`tuneGrid`) for key hyperparameters of each model (e.g., `k` in KNN, `mtry` in Random Forest, learning rate and tree depth in GBMs).
- **Sophisticated Feature Engineering:** Create interaction terms (e.g., `age * hypertension`, `bmi * avg_glucose_level`) to capture synergistic effects.
- **Cost-Sensitive Learning:** Instead of (or in addition to) SMOTE, directly incorporate the unequal costs of misclassification into the model's learning process (e.g., assign a higher penalty for misclassifying a "stroke" case as "non-stroke").

8. APPENDIX

```
---
title: "448_Project_Stroke_Data"
author: "Anurag Nepal"
date: "2025-03-16"
output:
  pdf_document: default
  html_document:
    df_print: paged
---
```

```
```{r setup, include=FALSE}
library(tidyverse)
library(ggplot2)
```
```

```
```{r}
stroke_data = read.csv("healthcare-dataset-stroke-data.csv")
```
```

```
```{r}
glimpse(stroke_data)
summary(stroke_data)
```
```

BMI should be integer but is there as character.

```
```{r}
stroke_data$bmi=as.double(stroke_data$bmi)
summary(stroke_data)
```
```

```
```{r}
stroke_counts = table(stroke_data$stroke)
stroke_counts_df = as.data.frame(stroke_counts)
stroke_counts_df$percentage = stroke_counts_df$Freq / sum(stroke_counts_df$Freq) * 100
colnames(stroke_counts_df) = c("stroke", "count", "percentage")
print(stroke_counts_df)
```
```

Data imbalance so may have to augment the data.

```
```{r}
na_counts = colSums(is.na(stroke_data))
na_counts[na_counts > 0]
```
```

```
```{r}
```

```
stroke_data = stroke_data[stroke_data$gender != "Other",]
```

```

```
```{r}
categorize_age = function(age) {
 if (age >= 0 & age <= 1) {
 return('Infant')
 } else if (age > 1 & age <= 12) {
 return('Children')
 } else if (age > 12 & age < 20) {
 return('Teenager')
 } else if (age >= 20 & age <= 39) {
 return('Adult')
 } else if (age > 39 & age <= 59) {
 return('Middle Aged')
 } else {
 return('Senior')
 }
}
```

```

```
stroke_data$age_group = sapply(stroke_data$age, categorize_age)
unique(stroke_data$age_group)
```

```

```
```{r}
avg_bmi_by_group = aggregate(bmi ~ age_group + gender, data = stroke_data, FUN = mean)
colnames(avg_bmi_by_group) = c("age_group", "gender", "avg_bmi")
avg_bmi_by_group
```

```

```
```{r}
stroke_data = merge(stroke_data, avg_bmi_by_group, by = c("age_group", "gender"), all.x = TRUE)
stroke_data$bmi = ifelse(is.na(stroke_data$bmi), stroke_data$avg_bmi, stroke_data$bmi)
stroke_data$avg_bmi = NULL
```

```

Used the average bmi for the age group and gender to fill in the bmi NA values.

```
```{r}
boxplot(age ~ stroke, data = stroke_data, main = "Age by Stroke Status", col = "skyblue")
boxplot(avg_glucose_level ~ stroke, data = stroke_data, main = "Glucose Level by Stroke Status",
col = "lightgreen")
boxplot(bmi ~ stroke, data = stroke_data, main = "BMI by Stroke Status", col = "salmon")
```

```

```

```{r}
barplot(sort(table(stroke_data$age_group)), beside = TRUE, main = "Distribution by Age Group",
        xlab = "Age Group", ylab = "Count", col = c("skyblue", "salmon"),
        legend = rownames(stroke_data$age_group))
age_group_stroke = prop.table((table(stroke_data$age_group, stroke_data$stroke)), margin = 1)
barplot(sort(age_group_stroke[, 2]), col = "skyblue", main = "Stroke Rate by Age Group", ylab =
"Stroke Proportion")
```

```{r}
hist(stroke_data$age, main = "Distribution of Age", xlab = "Age", col = "skyblue", breaks = 20,
prob = TRUE)
lines(density(stroke_data$age, na.rm = TRUE), col = "black", lwd = 3)
legend("topleft", legend = c("Histogram", "Density"), col = c("skyblue", "black"), lwd = 2)

plot(density(stroke_data$age[stroke_data$stroke == 1]), col = "red", main = "Age Distribution by
Stroke Status")
lines(density(stroke_data$age[stroke_data$stroke == 0]), col = "blue")
legend("topleft", legend = c("No Stroke", "Stroke"), fill = c("blue", "red"))
```

```{r}
hist(stroke_data$avg_glucose_level, main = "Distribution of Glucose Level", xlab = "Glucose
Level", col = "lightgreen", breaks = 20, prob = TRUE)
lines(density(stroke_data$avg_glucose_level), col = "black", lwd = 3)
legend("topright", legend = c("Histogram", "Density"), col = c("lightgreen", "black"), lwd = 2)
plot(density(stroke_data$avg_glucose_level[stroke_data$stroke == 0]), col = "blue", main =
"Glucose Level Distribution by Stroke Status")
lines(density(stroke_data$avg_glucose_level[stroke_data$stroke == 1]), col = "red")
legend("topright", legend = c("No Stroke", "Stroke"), fill = c("blue", "red"))
```

```{r}
hist(stroke_data$bmi, main = "Distribution of BMI", xlab = "BMI", col = "salmon", breaks =
20, prob = TRUE)
lines(density(stroke_data$bmi), col = "black", main = "BMI Distribution", lwd = 3)
legend("topright", legend = c("Histogram", "Density"), col = c("salmon", "black"), lwd = 2)
plot(density(stroke_data$bmi[stroke_data$stroke == 1]), col = "red", main = "BMI Distribution
by Stroke Status")
lines(density(stroke_data$bmi[stroke_data$stroke == 0]), col = "blue")
legend("topright", legend = c("No Stroke", "Stroke"), fill = c("blue", "red"))
```

```

Stroke likelihood increases with lower glucose concentration.

```

```{r}
barplot(table(stroke_data$gender), main = "Gender Distribution", col = "skyblue")
gender_stroke = prop.table(table(stroke_data$gender, stroke_data$stroke), margin = 1)

```

```
barplot(gender_stroke[, 2], col = "skyblue", main = "Stroke Rate by Gender", ylab = "Stroke Proportion")
```

```

```
```{r}
barplot(table(stroke_data$smoking_status), main = "Smoking Status Distribution", col = "orange")
smoke_stroke = prop.table(table(stroke_data$smoking_status, stroke_data$stroke), margin = 1)
barplot(smoke_stroke[, 2], col = "skyblue", main = "Stroke Rate by Smoking Status", ylab = "Stroke Proportion")
```

```

Former smokers have higher stroke rates.

```
```{r}
barplot(table(stroke_data$heart_disease), names.arg = c("No Heart Disease", "Heart Disease"),
main = "Heart Disease Distribution", col = "salmon")
hd_stroke = prop.table(table(stroke_data$heart_disease, stroke_data$stroke), margin = 1)
barplot(hd_stroke[, 2], names.arg = c("No Heart Disease", "Heart Disease"), col = "pink", main = "Stroke Rate by Heart Disease")
```

```

Heart disease increases stroke risk.

```
```{r}
stroke_data %>%
  group_by(work_type) %>%
  summarise(hypertension_rate = mean(hypertension)) %>%
  ggplot(aes(work_type, hypertension_rate)) +
  geom_col(fill = "orange") +
  labs(title = "Hypertension Rate by Work Type")
```

```

```
```{r}
stroke_data %>%
  group_by(ever_married) %>%
  summarise(hypertension_rate = mean(hypertension)) %>%
  ggplot(aes(ever_married, hypertension_rate)) +
  geom_col(fill = "lightblue") +
  labs(title = "Hypertension Rate by Marriage Status")
```

```

```
```{r}
stroke_data %>%
  group_by(Residence_type) %>%
  summarise(hypertension_rate = mean(hypertension)) %>%
  ggplot(aes(Residence_type, hypertension_rate)) +
  geom_col(fill = "lightblue") +

```

```

labs(title = "Hypertension Rate by Residence Type")
```

```{r}
barplot(table(stroke_data$hypertension), names.arg = c("No Hypertension", "Hypertension"),
main = "Hypertension Distribution", col = "lightgreen")
hypertension_stroke = prop.table(table(stroke_data$hypertension, stroke_data$stroke), margin =
1)
barplot(hypertension_stroke[, 2], names.arg = c("No Hypertension", "Hypertension"), col =
"skyblue", main = "Stroke Rate by Hypertension")
```

```{r}
barplot(table(stroke_data$work_type), main = "Work Type Distribution", col = "orange")
res_stroke = prop.table(table(stroke_data$work_type, stroke_data$stroke), margin = 1)
barplot(res_stroke[, 2], col = "salmon", main = "Stroke Rate by Work Type")
```

```{r}
barplot(table(stroke_data$Residence_type), main = "Residence Distribution", col = "orange")
res_stroke = prop.table(table(stroke_data$Residence_type, stroke_data$stroke), margin = 1)
barplot(res_stroke[, 2], names.arg = c("Rural", "Urban"), col = "salmon", main = "Stroke Rate by
Residence")
```

```{r}
barplot(table(stroke_data$ever_married), names.arg = c("Never Married", "Married"), main = "
Marital Status Distribution", col = "lightblue")
married_stroke = prop.table(table(stroke_data$ever_married, stroke_data$stroke), margin = 1)
barplot(married_stroke[, 2], names.arg = c("Never Married", "Married"), col = "purple", main =
"Stroke Rate by Marital Status")
```

```{r}
numerical_vars = stroke_data[, c("age", "avg_glucose_level", "bmi")]
numerical_vars$stroke = stroke_data$stroke
cor_matrix = cor(numerical_vars, use = "complete.obs")
print(cor_matrix)
```

```{r}
library(caret)
library(pROC)
```

```{r}

```

```

stroke_data = stroke_data %>%
  select(-id, -age_group, -Residence_type, -gender) %>%
  mutate(
    ever_married = as.factor(ever_married),
    work_type = as.factor(work_type),
    smoking_status = as.factor(smoking_status),
    hypertension = as.factor(hypertension),
    heart_disease = as.factor(heart_disease),
    stroke = as.factor(stroke)
  )
stroke_data = stroke_data %>%
  mutate(stroke = factor(stroke, levels = c(1, 0), labels = c("YES", "NO")))
```



```

```{r}
set.seed(123)
trainIndex = createDataPartition(stroke_data$stroke, p = 0.7, list = FALSE)
train_data = stroke_data[trainIndex,]
test_data = stroke_data[-trainIndex,]
prop.table(table(train_data$stroke))
prop.table(table(test_data$stroke))
```

```


```

## ## Define Training Controls

We'll define two `trainControl` objects for 5-fold cross-validation: one without SMOTE and one with SMOTE for class balancing. We'll use these across all models and scenarios.

```

```{r train-controls}
# Without SMOTE
ctrl_base = trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
)

ctrl_smote = trainControl(
  method = "cv",
  number = 5,
  classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final",
  sampling = "smote"
)

```

```
)  
...
```

For each model (Logistic Regression, LDA, QDA, KNN), we'll train under four scenarios:

1. ****No SMOTE, No PCA****
2. ****SMOTE, No PCA****
3. ****No SMOTE, PCA****
4. ****SMOTE, PCA****

Logistic Regression

```
```{r}  
No SMOTE, No PCA
set.seed(123)
logit_no_smote_no_pca = train(
 stroke ~ .,
 data = train_data,
 method = "glm",
 family = "binomial",
 trControl = ctrl_base,
 metric = "ROC"
)
logit_pred_no_smote_no_pca_prob = predict(logit_no_smote_no_pca, newdata = test_data, type
= "prob")
logit_pred_no_smote_no_pca_class = predict(logit_no_smote_no_pca, newdata = test_data, type
= "raw")
roc_logit_no_smote_no_pca = roc(test_data$stroke,
logit_pred_no_smote_no_pca_prob[, "YES"])
acc_logit_no_smote_no_pca = confusionMatrix(logit_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]
```
```

```
# SMOTE, No PCA  
set.seed(123)  
logit_smote_no_pca = train(  
  stroke ~ .,  
  data = train_data,  
  method = "glm",  
  family = "binomial",  
  trControl = ctrl_smote,  
  metric = "ROC"  
)
```

```

logit_pred_smote_no_pca_prob = predict(logit_smote_no_pca, newdata = test_data, type =
"prob")
logit_pred_smote_no_pca_class = predict(logit_smote_no_pca, newdata = test_data, type = "raw")
roc_logit_smote_no_pca = roc(test_data$stroke, logit_pred_smote_no_pca_prob[, "YES"])
acc_logit_smote_no_pca = confusionMatrix(logit_pred_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# No SMOTE, PCA
set.seed(123)
logit_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "glm",
  family = "binomial",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"), # PCA with centering and scaling
  metric = "ROC"
)
logit_pred_no_smote_pca_prob = predict(logit_no_smote_pca, newdata = test_data, type =
"prob")
logit_pred_no_smote_pca_class = predict(logit_no_smote_pca, newdata = test_data, type = "raw")
roc_logit_no_smote_pca = roc(test_data$stroke, logit_pred_no_smote_pca_prob[, "YES"])
acc_logit_no_smote_pca = confusionMatrix(logit_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# SMOTE, PCA
set.seed(123)
logit_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "glm",
  family = "binomial",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
logit_pred_smote_pca_prob = predict(logit_smote_pca, newdata = test_data, type = "prob")
roc_logit_smote_pca = roc(test_data$stroke, logit_pred_smote_pca_prob[, "YES"])
logit_pred_smote_pca_class = predict(logit_smote_pca, newdata = test_data, type = "raw")
acc_logit_smote_pca = confusionMatrix(logit_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# Plot ROC curves
plot(roc_logit_no_smote_no_pca, col = "blue", main = "ROC for Logistic Regression")

```

```

lines(roc_logit_smote_no_pca, col = "red")
lines(roc_logit_no_smote_pca, col = "green")
lines(roc_logit_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
```

```
```{r}
Linear Discriminant Analysis (LDA)
No SMOTE, No PCA
set.seed(123)
lda_no_smote_no_pca = train(
 stroke ~ .,
 data = train_data,
 method = "lda",
 trControl = ctrl_base,
 metric = "ROC"
)
lda_pred_no_smote_no_pca = predict(lda_no_smote_no_pca, newdata = test_data, type = "prob")
roc_lda_no_smote_no_pca = roc(test_data$stroke, lda_pred_no_smote_no_pca[, "YES"])
lda_pred_no_smote_no_pca_class = predict(lda_no_smote_no_pca, newdata = test_data, type =
"raw")
acc_lda_no_smote_no_pca = confusionMatrix(lda_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

SMOTE, No PCA
set.seed(123)
lda_smote_no_pca = train(
 stroke ~ .,
 data = train_data,
 method = "lda",
 trControl = ctrl_smote,
 metric = "ROC"
)
lda_pred_smote_no_pca = predict(lda_smote_no_pca, newdata = test_data, type = "prob")
roc_lda_smote_no_pca = roc(test_data$stroke, lda_pred_smote_no_pca[, "YES"])
lda_pred_smote_no_pca_class = predict(lda_smote_no_pca, newdata = test_data, type = "raw")
acc_lda_smote_no_pca = confusionMatrix(lda_pred_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

No SMOTE, PCA
set.seed(123)
lda_no_smote_pca = train(
 stroke ~ .,
 data = train_data,
 method = "lda",
 trControl = ctrl_base,
 preProcess = c("center", "scale", "pca"),
 metric = "ROC"
)
lda_pred_no_smote_pca = predict(lda_no_smote_pca, newdata = test_data, type = "prob")
roc_lda_no_smote_pca = roc(test_data$stroke, lda_pred_no_smote_pca[, "YES"])
lda_pred_no_smote_pca_class = predict(lda_no_smote_pca, newdata = test_data, type = "raw")
acc_lda_no_smote_pca = confusionMatrix(lda_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

SMOTE, PCA
set.seed(123)
lda_smote_pca = train(
 stroke ~ .,
 data = train_data,
 method = "lda",
 trControl = ctrl_smote,
 preProcess = c("center", "scale", "pca"),
 metric = "ROC"
)
lda_pred_smote_pca = predict(lda_smote_pca, newdata = test_data, type = "prob")
roc_lda_smote_pca = roc(test_data$stroke, lda_pred_smote_pca[, "YES"])
lda_pred_smote_pca_class = predict(lda_smote_pca, newdata = test_data, type = "raw")
acc_lda_smote_pca = confusionMatrix(lda_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]
Plot ROC curves
plot(roc_lda_no_smote_no_pca, col = "blue", main = "ROC for LDA")
lines(roc_lda_smote_no_pca, col = "red")
lines(roc_lda_no_smote_pca, col = "green")
lines(roc_lda_smote_pca, col = "purple")
legend("bottomright",
 legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
 col = c("blue", "red", "green", "purple"),
 lty = 1)
...

```

```

``{r}

```

### ### Quadratic Discriminant Analysis (QDA)

# No SMOTE, No PCA

set.seed(123)

qda\_no\_smote\_no\_pca = tryCatch({

  train(

    stroke ~ .,

    data = train\_data,

    method = "qda",

    trControl = ctrl\_base,

    metric = "ROC"

  )

}, error = function(e) NULL)

qda\_pred\_no\_smote\_no\_pca = if (!is.null(qda\_no\_smote\_no\_pca)) {

  predict(qda\_no\_smote\_no\_pca, newdata = test\_data, type = "prob")

} else {

  NULL

}

roc\_qda\_no\_smote\_no\_pca = if (!is.null(qda\_pred\_no\_smote\_no\_pca)) {

  roc(test\_data\$stroke, qda\_pred\_no\_smote\_no\_pca[, "YES"])

} else {

  NULL

}

qda\_pred\_no\_smote\_no\_pca\_class = if (!is.null(qda\_no\_smote\_no\_pca)) {

  predict(qda\_no\_smote\_no\_pca, newdata = test\_data, type = "raw")

} else {

  NULL

}

acc\_qda\_no\_smote\_no\_pca = if (!is.null(qda\_pred\_no\_smote\_no\_pca\_class)) {

  confusionMatrix(qda\_pred\_no\_smote\_no\_pca\_class, test\_data\$stroke)\$overall["Accuracy"]

} else {

  NA

}

# SMOTE, No PCA

set.seed(123)

qda\_smote\_no\_pca = tryCatch({

  train(

    stroke ~ .,

    data = train\_data,

    method = "qda",

    trControl = ctrl\_smote,

    metric = "ROC"

  )

}, error = function(e) NULL)

```

qda_pred_smote_no_pca = if (!is.null(qda_smote_no_pca)) {
 predict(qda_smote_no_pca, newdata = test_data, type = "prob")
} else {
 NULL
}
roc_qda_smote_no_pca = if (!is.null(qda_pred_smote_no_pca)) {
 roc(test_data$stroke, qda_pred_smote_no_pca[, "YES"])
} else {
 NULL
}
qda_pred_smote_no_pca_class = if (!is.null(qda_smote_no_pca)) {
 predict(qda_smote_no_pca, newdata = test_data, type = "raw")
} else {
 NULL
}
acc_qda_smote_no_pca = if (!is.null(qda_pred_smote_no_pca_class)) {
 confusionMatrix(qda_pred_smote_no_pca_class, test_data$stroke)$overall["Accuracy"]
} else {
 NA
}

No SMOTE, PCA
set.seed(123)
qda_no_smote_pca = train(
 stroke ~ .,
 data = train_data,
 method = "qda",
 trControl = ctrl_base,
 preProcess = c("center", "scale", "pca"),
 metric = "ROC"
)
qda_pred_no_smote_pca = predict(qda_no_smote_pca, newdata = test_data, type = "prob")
roc_qda_no_smote_pca = roc(test_data$stroke, qda_pred_no_smote_pca[, "YES"])
qda_pred_no_smote_pca_class = predict(qda_no_smote_pca, newdata = test_data, type = "raw")
acc_qda_no_smote_pca = confusionMatrix(qda_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

SMOTE, PCA
set.seed(123)
qda_smote_pca = train(
 stroke ~ .,
 data = train_data,
 method = "qda",
 trControl = ctrl_smote,
 preProcess = c("center", "scale", "pca"),

```

```

metric = "ROC"
)
qda_pred_smote_pca = predict(qda_smote_pca, newdata = test_data, type = "prob")
roc_qda_smote_pca = roc(test_data$stroke, qda_pred_smote_pca[, "YES"])
qda_pred_smote_pca_class = predict(qda_smote_pca, newdata = test_data, type = "raw")
acc_qda_smote_pca = confusionMatrix(qda_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

Plot ROC curves (only for successful models)
plot(roc_qda_no_smote_pca, col = "green", main = "ROC for QDA")
lines(roc_qda_smote_pca, col = "purple")
if (!is.null(roc_qda_no_smote_no_pca)) lines(roc_qda_no_smote_no_pca, col = "blue")
if (!is.null(roc_qda_smote_no_pca)) lines(roc_qda_smote_no_pca, col = "red")
legend("bottomright",
 legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA")[c(!is.null(roc_qda_no_smote_no_pca), !is.null(roc_qda_smote_no_pca), TRUE, TRUE)],
 col = c("blue", "red", "green", "purple")[c(!is.null(roc_qda_no_smote_no_pca),
!is.null(roc_qda_smote_no_pca), TRUE, TRUE)],
 lty = 1)

...

```{r}

#### K-Nearest Neighbors (KNN)

# No SMOTE, No PCA
set.seed(123)
knn_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_base,
  metric = "ROC"
)
knn_pred_no_smote_no_pca = predict(knn_no_smote_no_pca, newdata = test_data, type =
"prob")
roc_knn_no_smote_no_pca = roc(test_data$stroke, knn_pred_no_smote_no_pca[, "YES"])
knn_pred_no_smote_no_pca_class = predict(knn_no_smote_no_pca, newdata = test_data, type =
"raw")
acc_knn_no_smote_no_pca = confusionMatrix(knn_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

# SMOTE, No PCA

```

```

set.seed(123)
knn_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_smote,
  metric = "ROC"
)
knn_pred_smote_no_pca = predict(knn_smote_no_pca, newdata = test_data, type = "prob")
roc_knn_smote_no_pca = roc(test_data$stroke, knn_pred_smote_no_pca[, "YES"])
knn_pred_smote_no_pca_class = predict(knn_smote_no_pca, newdata = test_data, type = "raw")
acc_knn_smote_no_pca = confusionMatrix(knn_pred_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

No SMOTE, PCA

```

set.seed(123)
knn_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
knn_pred_no_smote_pca = predict(knn_no_smote_pca, newdata = test_data, type = "prob")
roc_knn_no_smote_pca = roc(test_data$stroke, knn_pred_no_smote_pca[, "YES"])
knn_pred_no_smote_pca_class = predict(knn_no_smote_pca, newdata = test_data, type = "raw")
acc_knn_no_smote_pca = confusionMatrix(knn_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

```

SMOTE, PCA

```

set.seed(123)
knn_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "knn",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  metric = "ROC"
)
knn_pred_smote_pca = predict(knn_smote_pca, newdata = test_data, type = "prob")
roc_knn_smote_pca = roc(test_data$stroke, knn_pred_smote_pca[, "YES"])
knn_pred_smote_pca_class = predict(knn_smote_pca, newdata = test_data, type = "raw")
acc_knn_smote_pca = confusionMatrix(knn_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# Plot ROC curves
plot(roc_knn_no_smote_no_pca, col = "blue", main = "ROC for KNN")
lines(roc_knn_smote_no_pca, col = "red")
lines(roc_knn_no_smote_pca, col = "green")
lines(roc_knn_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
...

```

```

```{r}
library(gbm) # For boosting (gbm)
library(rpart) # For decision trees
library(ipred) # For bagging
library(tibble) # For AUC data frame
library(dplyr) # For data manipulation
library(knitr) # For table formatting
...

```

```

```{r}
#Decision Tree
# No SMOTE, No PCA
set.seed(123)
dt_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_base,
  tuneLength = 5, # Tune complexity parameter
  metric = "ROC"
)
dt_pred_no_smote_no_pca_prob = predict(dt_no_smote_no_pca, newdata = test_data, type =
"prob")
dt_pred_no_smote_no_pca_class = predict(dt_no_smote_no_pca, newdata = test_data, type =
"raw")
roc_dt_no_smote_no_pca = roc(test_data$stroke, dt_pred_no_smote_no_pca_prob[, "YES"])
acc_dt_no_smote_no_pca = confusionMatrix(dt_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# SMOTE, No PCA
set.seed(123)

```

```

dt_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_smote,
  tuneLength = 5,
  metric = "ROC"
)
dt_pred_smote_no_pca_prob = predict(dt_smote_no_pca, newdata = test_data, type = "prob")
dt_pred_smote_no_pca_class = predict(dt_smote_no_pca, newdata = test_data, type = "raw")
roc_dt_smote_no_pca = roc(test_data$stroke, dt_pred_smote_no_pca_prob[, "YES"])
acc_dt_smote_no_pca = confusionMatrix(dt_pred_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# No SMOTE, PCA
set.seed(123)
dt_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 5,
  metric = "ROC"
)
dt_pred_no_smote_pca_prob = predict(dt_no_smote_pca, newdata = test_data, type = "prob")
dt_pred_no_smote_pca_class = predict(dt_no_smote_pca, newdata = test_data, type = "raw")
roc_dt_no_smote_pca = roc(test_data$stroke, dt_pred_no_smote_pca_prob[, "YES"])
acc_dt_no_smote_pca = confusionMatrix(dt_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

# SMOTE, PCA
set.seed(123)
dt_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rpart",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 5,
  metric = "ROC"
)
dt_pred_smote_pca_prob = predict(dt_smote_pca, newdata = test_data, type = "prob")

```

```
dt_pred_smote_pca_class = predict(dt_smote_pca, newdata = test_data, type = "raw")
roc_dt_smote_pca = roc(test_data$stroke, dt_pred_smote_pca_prob[, "YES"])
acc_dt_smote_pca = confusionMatrix(dt_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]
```

```
# Plot ROC curves
plot(roc_dt_no_smote_no_pca, col = "blue", main = "ROC for Decision Tree")
lines(roc_dt_smote_no_pca, col = "red")
lines(roc_dt_no_smote_pca, col = "green")
lines(roc_dt_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
...`
```

```
{r}
#Random Forest
# No SMOTE, No PCA
set.seed(123)
rf_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl_base,
  tuneLength = 3, # Tune mtry
  metric = "ROC"
)
rf_pred_no_smote_no_pca = predict(rf_no_smote_no_pca, newdata = test_data, type = "prob")
roc_rf_no_smote_no_pca = roc(test_data$stroke, rf_pred_no_smote_no_pca[, "YES"])
rf_pred_no_smote_no_pca_class = predict(rf_no_smote_no_pca, newdata = test_data, type =
"raw")
acc_rf_no_smote_no_pca = confusionMatrix(rf_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]
```

```
# SMOTE, No PCA
set.seed(123)
rf_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl_smote,
  tuneLength = 3,
```

```

    metric = "ROC"
  )
  rf_pred_smote_no_pca = predict(rf_smote_no_pca, newdata = test_data, type = "prob")
  roc_rf_smote_no_pca = roc(test_data$stroke, rf_pred_smote_no_pca[, "YES"])
  rf_pred_smote_no_pca_class = predict(rf_smote_no_pca, newdata = test_data, type = "raw")
  acc_rf_smote_no_pca = confusionMatrix(rf_pred_smote_no_pca_class,
    test_data$stroke)$overall["Accuracy"]

# No SMOTE, PCA
set.seed(123)
rf_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 3,
  metric = "ROC"
)
rf_pred_no_smote_pca = predict(rf_no_smote_pca, newdata = test_data, type = "prob")
roc_rf_no_smote_pca = roc(test_data$stroke, rf_pred_no_smote_pca[, "YES"])
rf_pred_no_smote_pca_class = predict(rf_no_smote_pca, newdata = test_data, type = "raw")
acc_rf_no_smote_pca = confusionMatrix(rf_pred_no_smote_pca_class,
  test_data$stroke)$overall["Accuracy"]

# SMOTE, PCA
set.seed(123)
rf_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "rf",
  trControl = ctrl_smote,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 3,
  metric = "ROC"
)
rf_pred_smote_pca = predict(rf_smote_pca, newdata = test_data, type = "prob")
roc_rf_smote_pca = roc(test_data$stroke, rf_pred_smote_pca[, "YES"])
rf_pred_smote_pca_class = predict(rf_smote_pca, newdata = test_data, type = "raw")
acc_rf_smote_pca = confusionMatrix(rf_pred_smote_pca_class,
  test_data$stroke)$overall["Accuracy"]

# Plot ROC curves
plot(roc_rf_no_smote_no_pca, col = "blue", main = "ROC for Random Forest")
lines(roc_rf_smote_no_pca, col = "red")

```

```

lines(roc_rf_no_smote_pca, col = "green")
lines(roc_rf_smote_pca, col = "purple")
legend("bottomright",
      legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
      col = c("blue", "red", "green", "purple"),
      lty = 1)
...

```

```

```{r}
#Bagging
No SMOTE, No PCA
set.seed(123)
bag_no_smote_no_pca = train(
 stroke ~ .,
 data = train_data,
 method = "treebag",
 trControl = ctrl_base,
 metric = "ROC"
)
bag_pred_no_smote_no_pca = predict(bag_no_smote_no_pca, newdata = test_data, type =
"prob")
roc_bag_no_smote_no_pca = roc(test_data$stroke, bag_pred_no_smote_no_pca[, "YES"])
bag_pred_no_smote_no_pca_class = predict(bag_no_smote_no_pca, newdata = test_data, type =
"raw")
acc_bag_no_smote_no_pca = confusionMatrix(bag_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

SMOTE, No PCA
set.seed(123)
bag_smote_no_pca = train(
 stroke ~ .,
 data = train_data,
 method = "treebag",
 trControl = ctrl_smote,
 metric = "ROC"
)
bag_pred_smote_no_pca = predict(bag_smote_no_pca, newdata = test_data, type = "prob")
roc_bag_smote_no_pca = roc(test_data$stroke, bag_pred_smote_no_pca[, "YES"])
bag_pred_smote_no_pca_class = predict(bag_smote_no_pca, newdata = test_data, type = "raw")
acc_bag_smote_no_pca = confusionMatrix(bag_pred_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

```

No SMOTE, PCA
set.seed(123)

```

```

bag_no_smote_pca = train(
 stroke ~ .,
 data = train_data,
 method = "treebag",
 trControl = ctrl_base,
 preProcess = c("center", "scale", "pca"),
 metric = "ROC"
)
bag_pred_no_smote_pca = predict(bag_no_smote_pca, newdata = test_data, type = "prob")
roc_bag_no_smote_pca = roc(test_data$stroke, bag_pred_no_smote_pca[, "YES"])
bag_pred_no_smote_pca_class = predict(bag_no_smote_pca, newdata = test_data, type = "raw")
acc_bag_no_smote_pca = confusionMatrix(bag_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

SMOTE, PCA
set.seed(123)
bag_smote_pca = train(
 stroke ~ .,
 data = train_data,
 method = "treebag",
 trControl = ctrl_smote,
 preProcess = c("center", "scale", "pca"),
 metric = "ROC"
)
bag_pred_smote_pca = predict(bag_smote_pca, newdata = test_data, type = "prob")
roc_bag_smote_pca = roc(test_data$stroke, bag_pred_smote_pca[, "YES"])
bag_pred_smote_pca_class = predict(bag_smote_pca, newdata = test_data, type = "raw")
acc_bag_smote_pca = confusionMatrix(bag_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

Plot ROC curves
plot(roc_bag_no_smote_no_pca, col = "blue", main = "ROC for Bagging")
lines(roc_bag_smote_no_pca, col = "red")
lines(roc_bag_no_smote_pca, col = "green")
lines(roc_bag_smote_pca, col = "purple")
legend("bottomright",
 legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
 col = c("blue", "red", "green", "purple"),
 lty = 1)
...

```{r}
#Boosting
# No SMOTE, No PCA

```

```

set.seed(123)
boost_no_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "gbm",
  trControl = ctrl_base,
  tuneLength = 3, # Tune n.trees, interaction.depth, etc.
  metric = "ROC",
  verbose = FALSE
)
boost_pred_no_smote_no_pca = predict(boost_no_smote_no_pca, newdata = test_data, type =
"prob")
roc_boost_no_smote_no_pca = roc(test_data$stroke, boost_pred_no_smote_no_pca[, "YES"])
boost_pred_no_smote_no_pca_class = predict(boost_no_smote_no_pca, newdata = test_data,
type = "raw")
acc_boost_no_smote_no_pca = confusionMatrix(boost_pred_no_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

SMOTE, No PCA

```

set.seed(123)
boost_smote_no_pca = train(
  stroke ~ .,
  data = train_data,
  method = "gbm",
  trControl = ctrl_smote,
  tuneLength = 3,
  metric = "ROC",
  verbose = FALSE
)
boost_pred_smote_no_pca = predict(boost_smote_no_pca, newdata = test_data, type = "prob")
roc_boost_smote_no_pca = roc(test_data$stroke, boost_pred_smote_no_pca[, "YES"])
boost_pred_smote_no_pca_class = predict(boost_smote_no_pca, newdata = test_data, type =
"raw")
acc_boost_smote_no_pca = confusionMatrix(boost_pred_smote_no_pca_class,
test_data$stroke)$overall["Accuracy"]

```

No SMOTE, PCA

```

set.seed(123)
boost_no_smote_pca = train(
  stroke ~ .,
  data = train_data,
  method = "gbm",
  trControl = ctrl_base,
  preProcess = c("center", "scale", "pca"),
  tuneLength = 3,

```

```

metric = "ROC",
verbose = FALSE
)
boost_pred_no_smote_pca = predict(boost_no_smote_pca, newdata = test_data, type = "prob")
roc_boost_no_smote_pca = roc(test_data$stroke, boost_pred_no_smote_pca[, "YES"])
boost_pred_no_smote_pca_class = predict(boost_no_smote_pca, newdata = test_data, type =
"raw")
acc_boost_no_smote_pca = confusionMatrix(boost_pred_no_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

# SMOTE, PCA
set.seed(123)
boost_smote_pca = train(
stroke ~ .,
data = train_data,
method = "gbm",
trControl = ctrl_smote,
preProcess = c("center", "scale", "pca"),
tuneLength = 3,
metric = "ROC",
verbose = FALSE
)
boost_pred_smote_pca = predict(boost_smote_pca, newdata = test_data, type = "prob")
roc_boost_smote_pca = roc(test_data$stroke, boost_pred_smote_pca[, "YES"])
boost_pred_smote_pca_class = predict(boost_smote_pca, newdata = test_data, type = "raw")
acc_boost_smote_pca = confusionMatrix(boost_pred_smote_pca_class,
test_data$stroke)$overall["Accuracy"]

# Plot ROC curves
plot(roc_boost_no_smote_no_pca, col = "blue", main = "ROC for Boosting")
lines(roc_boost_smote_no_pca, col = "red")
lines(roc_boost_no_smote_pca, col = "green")
lines(roc_boost_smote_pca, col = "purple")
legend("bottomright",
legend = c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"),
col = c("blue", "red", "green", "purple"),
lty = 1)
...
```{r}
sens_logit_no_smote_no_pca = confusionMatrix(logit_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_logit_smote_no_pca = confusionMatrix(logit_pred_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]

```

```

sens_logit_no_smote_pca = confusionMatrix(logit_pred_no_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_logit_smote_pca = confusionMatrix(logit_pred_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_no_pca = confusionMatrix(lda_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_no_pca = confusionMatrix(lda_pred_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_pca = confusionMatrix(lda_pred_no_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_pca = confusionMatrix(lda_pred_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_no_pca = confusionMatrix(lda_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_no_pca = confusionMatrix(lda_pred_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_no_smote_pca = confusionMatrix(lda_pred_no_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_lda_smote_pca = confusionMatrix(lda_pred_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_qda_no_smote_no_pca = if (!is.null(qda_pred_no_smote_no_pca_class)) {
 confusionMatrix(qda_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
} else {
 NA
}
sens_qda_smote_no_pca = if (!is.null(qda_pred_smote_no_pca_class)) {
 confusionMatrix(qda_pred_smote_no_pca_class, test_data$stroke)$byClass["Sensitivity"]
} else {
 NA
}
sens_qda_no_smote_pca = confusionMatrix(qda_pred_no_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_qda_smote_pca = confusionMatrix(qda_pred_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_dt_no_smote_no_pca = confusionMatrix(dt_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_dt_smote_no_pca = confusionMatrix(dt_pred_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_dt_no_smote_pca = confusionMatrix(dt_pred_no_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_dt_smote_pca = confusionMatrix(dt_pred_smote_pca_class,
test_data$stroke)$byClass["Sensitivity"]
sens_rf_no_smote_no_pca = confusionMatrix(rf_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Sensitivity"]

```

```

sens_rf_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_rf_no_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_rf_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_bag_no_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_bag_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_bag_no_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_bag_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_boost_no_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_boost_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_boost_no_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_boost_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_knn_no_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_knn_smote_no_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_knn_no_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
sens_knn_smote_pca =
test_data$stroke)$byClass["Sensitivity"]
...
```{r}
spec_logit_no_smote_no_pca    =
test_data$stroke)$byClass["Specificity"]
spec_logit_smote_no_pca       =
test_data$stroke)$byClass["Specificity"]
spec_logit_no_smote_pca       =
test_data$stroke)$byClass["Specificity"]
spec_logit_smote_pca          =
test_data$stroke)$byClass["Specificity"]
spec_lda_no_smote_no_pca      =
test_data$stroke)$byClass["Specificity"]
spec_lda_smote_no_pca         =
test_data$stroke)$byClass["Specificity"]

```

```

confusionMatrix(rf_pred_smote_no_pca_class,
confusionMatrix(rf_pred_no_smote_pca_class,
confusionMatrix(rf_pred_smote_pca_class,
confusionMatrix(bag_pred_no_smote_no_pca_class,
confusionMatrix(bag_pred_smote_no_pca_class,
confusionMatrix(bag_pred_no_smote_pca_class,
confusionMatrix(bag_pred_smote_pca_class,
confusionMatrix(boost_pred_no_smote_no_pca_class,
confusionMatrix(boost_pred_smote_no_pca_class,
confusionMatrix(boost_pred_no_smote_pca_class,
confusionMatrix(boost_pred_smote_pca_class,
confusionMatrix(knn_pred_no_smote_no_pca_class,
confusionMatrix(knn_pred_smote_no_pca_class,
confusionMatrix(knn_pred_no_smote_pca_class,
confusionMatrix(knn_pred_smote_pca_class,
confusionMatrix(logit_pred_no_smote_no_pca_class,
confusionMatrix(logit_pred_smote_no_pca_class,
confusionMatrix(logit_pred_no_smote_pca_class,
confusionMatrix(logit_pred_smote_pca_class,
confusionMatrix(lda_pred_no_smote_no_pca_class,
confusionMatrix(lda_pred_smote_no_pca_class,

```

```

spec_lda_no_smote_pca          =          confusionMatrix(lda_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_lda_smote_pca             =          confusionMatrix(lda_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_lda_no_smote_no_pca       =          confusionMatrix(lda_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_lda_smote_no_pca          =          confusionMatrix(lda_pred_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_lda_no_smote_pca          =          confusionMatrix(lda_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_lda_smote_pca             =          confusionMatrix(lda_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_qda_no_smote_no_pca = if (!is.null(qda_pred_no_smote_no_pca_class)) {
  confusionMatrix(qda_pred_no_smote_no_pca_class, test_data$stroke)$byClass["Specificity"]
} else {
  NA
}
spec_qda_smote_no_pca = if (!is.null(qda_pred_smote_no_pca_class)) {
  confusionMatrix(qda_pred_smote_no_pca_class, test_data$stroke)$byClass["Specificity"]
} else {
  NA
}
spec_qda_no_smote_pca          =          confusionMatrix(qda_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_qda_smote_pca             =          confusionMatrix(qda_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_dt_no_smote_no_pca       =          confusionMatrix(dt_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_dt_smote_no_pca           =          confusionMatrix(dt_pred_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_dt_no_smote_pca           =          confusionMatrix(dt_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_dt_smote_pca              =          confusionMatrix(dt_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_rf_no_smote_no_pca        =          confusionMatrix(rf_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_rf_smote_no_pca           =          confusionMatrix(rf_pred_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_rf_no_smote_pca           =          confusionMatrix(rf_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_rf_smote_pca              =          confusionMatrix(rf_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_bag_no_smote_no_pca       =          confusionMatrix(bag_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]

```

```

spec_bag_smote_no_pca          = confusionMatrix(bag_pred_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_bag_no_smote_pca          = confusionMatrix(bag_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_bag_smote_pca              = confusionMatrix(bag_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_boost_no_smote_no_pca      = confusionMatrix(boost_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_boost_smote_no_pca         = confusionMatrix(boost_pred_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_boost_no_smote_pca         = confusionMatrix(boost_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_boost_smote_pca            = confusionMatrix(boost_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_knn_no_smote_no_pca        = confusionMatrix(knn_pred_no_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_knn_smote_no_pca           = confusionMatrix(knn_pred_smote_no_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_knn_no_smote_pca           = confusionMatrix(knn_pred_no_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
spec_knn_smote_pca              = confusionMatrix(knn_pred_smote_pca_class,
test_data$stroke)$byClass["Specificity"]
```


```

```{r}
results_df = data.frame(
 Model = rep(c("Logistic Regression", "LDA", "QDA", "Decision Tree",
 "Random Forest", "Bagging", "Boosting", "KNN"), each = 4),
 Scenario = rep(c("No SMOTE No PCA", "SMOTE No PCA", "No SMOTE PCA", "SMOTE
PCA"), 8),
 AUC = c(
 auc(roc_logit_no_smote_no_pca), auc(roc_logit_smote_no_pca),
 auc(roc_logit_no_smote_pca), auc(roc_logit_smote_pca),
 auc(roc_lda_no_smote_no_pca), auc(roc_lda_smote_no_pca), auc(roc_lda_no_smote_pca),
 auc(roc_lda_smote_pca),
 if (!is.null(roc_qda_no_smote_no_pca)) auc(roc_qda_no_smote_no_pca) else NA,
 if (!is.null(roc_qda_smote_no_pca)) auc(roc_qda_smote_no_pca) else NA,
 auc(roc_qda_no_smote_pca), auc(roc_qda_smote_pca),
 auc(roc_dt_no_smote_no_pca), auc(roc_dt_smote_no_pca), auc(roc_dt_no_smote_pca),
 auc(roc_dt_smote_pca),
 auc(roc_rf_no_smote_no_pca), auc(roc_rf_smote_no_pca), auc(roc_rf_no_smote_pca),
 auc(roc_rf_smote_pca),
 auc(roc_bag_no_smote_no_pca), auc(roc_bag_smote_no_pca), auc(roc_bag_no_smote_pca),
 auc(roc_bag_smote_pca),
 auc(roc_boost_no_smote_no_pca), auc(roc_boost_smote_no_pca),
 auc(roc_boost_no_smote_pca), auc(roc_boost_smote_pca),

```


```

```

auc(roc_knn_no_smote_no_pca), auc(roc_knn_smote_no_pca), auc(roc_knn_no_smote_pca),
auc(roc_knn_smote_pca)
),
Accuracy = c(
  acc_logit_no_smote_no_pca,      acc_logit_smote_no_pca,      acc_logit_no_smote_pca,
  acc_logit_smote_pca,
  acc_lda_no_smote_no_pca,        acc_lda_smote_no_pca,        acc_lda_no_smote_pca,
  acc_lda_smote_pca,
  acc_qda_no_smote_no_pca,        acc_qda_smote_no_pca,        acc_qda_no_smote_pca,
  acc_qda_smote_pca,
  acc_dt_no_smote_no_pca, acc_dt_smote_no_pca, acc_dt_no_smote_pca, acc_dt_smote_pca,
  acc_rf_no_smote_no_pca, acc_rf_smote_no_pca, acc_rf_no_smote_pca, acc_rf_smote_pca,
  acc_bag_no_smote_no_pca,        acc_bag_smote_no_pca,        acc_bag_no_smote_pca,
  acc_bag_smote_pca,
  acc_boost_no_smote_no_pca,      acc_boost_smote_no_pca,      acc_boost_no_smote_pca,
  acc_boost_smote_pca,
  acc_knn_no_smote_no_pca,        acc_knn_smote_no_pca,        acc_knn_no_smote_pca,
  acc_knn_smote_pca
),
Sensitivity = c(
  sens_logit_no_smote_no_pca,      sens_logit_smote_no_pca,      sens_logit_no_smote_pca,
  sens_logit_smote_pca,
  sens_lda_no_smote_no_pca,        sens_lda_smote_no_pca,        sens_lda_no_smote_pca,
  sens_lda_smote_pca,
  sens_qda_no_smote_no_pca,        sens_qda_smote_no_pca,        sens_qda_no_smote_pca,
  sens_qda_smote_pca,
  sens_dt_no_smote_no_pca,        sens_dt_smote_no_pca,        sens_dt_no_smote_pca,
  sens_dt_smote_pca,
  sens_rf_no_smote_no_pca,        sens_rf_smote_no_pca,        sens_rf_no_smote_pca,
  sens_rf_smote_pca,
  sens_bag_no_smote_no_pca,        sens_bag_smote_no_pca,        sens_bag_no_smote_pca,
  sens_bag_smote_pca,
  sens_boost_no_smote_no_pca,      sens_boost_smote_no_pca,      sens_boost_no_smote_pca,
  sens_boost_smote_pca,
  sens_knn_no_smote_no_pca,        sens_knn_smote_no_pca,        sens_knn_no_smote_pca,
  sens_knn_smote_pca
),
Specificity = c(
  spec_logit_no_smote_no_pca,      spec_logit_smote_no_pca,      spec_logit_no_smote_pca,
  spec_logit_smote_pca,
  spec_lda_no_smote_no_pca,        spec_lda_smote_no_pca,        spec_lda_no_smote_pca,
  spec_lda_smote_pca,
  spec_qda_no_smote_no_pca,        spec_qda_smote_no_pca,        spec_qda_no_smote_pca,
  spec_qda_smote_pca,

```

```

spec_dt_no_smote_no_pca,      spec_dt_smote_no_pca,      spec_dt_no_smote_pca,
spec_dt_smote_pca,
spec_rf_no_smote_no_pca,      spec_rf_smote_no_pca,      spec_rf_no_smote_pca,
spec_rf_smote_pca,
spec_bag_no_smote_no_pca,     spec_bag_smote_no_pca,     spec_bag_no_smote_pca,
spec_bag_smote_pca,
spec_boost_no_smote_no_pca,   spec_boost_smote_no_pca,   spec_boost_no_smote_pca,
spec_boost_smote_pca,
spec_knn_no_smote_no_pca,     spec_knn_smote_no_pca,     spec_knn_no_smote_pca,
spec_knn_smote_pca
)
) %>%
  filter(!is.na(AUC) & !is.na(Accuracy) & !is.na(Specificity))

```

```

kable(results_df, caption = "AUC, Accuracy, Sensitivity, and Specificity for Each Model and Scenario (NA Removed)", digits = 4)
```

```

```

```{r}
results_df$Scenario = factor(results_df$Scenario,
                             levels = c("No SMOTE No PCA", "SMOTE No PCA",
                                           "No SMOTE PCA", "SMOTE PCA"))

ggplot(results_df, aes(x = Model, y = Sensitivity, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.45)) +
  labs(title = "Sensitivity Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "Sensitivity") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")
```

```

```

```{r}
results_df$Scenario = factor(results_df$Scenario,
                             levels = c("No SMOTE No PCA", "SMOTE No PCA",
                                           "No SMOTE PCA", "SMOTE PCA"))

ggplot(results_df, aes(x = Model, y = Specificity, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.45)) +
  labs(title = "Specificity Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "Specificity") +
  theme_minimal() +

```

```

theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
scale_fill_brewer(palette = "Set1")
```

```

Because the QDA model fails to fit in every fold due to the rank deficiency issue, no predictions are produced. Without predictions, the performance metrics (like ROC) cannot be computed, leading to the "missing values" warning.

```

```{r}
# Convert Scenario to factor for consistent ordering
results_df$Scenario = factor(results_df$Scenario,
                             levels = c("No SMOTE No PCA", "SMOTE No PCA",
                                           "No SMOTE PCA", "SMOTE PCA"))

# Create grouped bar plot
ggplot(results_df, aes(x = Model, y = AUC, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.9)) +
  labs(title = "AUC Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "AUC") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")
```

```

```

```{r}
# Accuracy Plot
ggplot(results_df, aes(x = Model, y = Accuracy, fill = Scenario)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.45)) +
  labs(title = "Accuracy Comparison Across Models and Scenarios (NA Removed)",
       x = "Model",
       y = "Accuracy") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")
```

```